TECHNICAL DOCUMENTATION · INTERVIEW REFERENCE

NFL Season Hub

Six NFL seasons you can drill through — league standings, any team, any game — ending in a scrubbable, 60fps canvas replay of how that game's win probability actually moved.

1,695 GAMES WITH REPLAYS	**273,623** PLAYS PROCESSED	**4,166** STATIC JSON FILES	**0** BACKEND SERVICES
389 UNIT / COMPONENT TESTS	**40×2** E2E SPECS × DEVICES	**100** LIGHTHOUSE, DESKTOP, ALL ROUTES	**0.14ms** CANVAS DRAW PER FRAME

React 19 · TypeScript · Vite 8 · Tailwind v4 · Canvas 2D · Python + polars ETL · Playwright · Vitest · GitHub Actions · Vercel

Live: nfl-season-hub.vercel.app | Repo: github.com/Rakesh0710/nfl-season-hub | Data: nflverse, CC BY 4.0

Contents

01 **The 90-second pitch**
What it is, why it exists, and the one sentence to open with

02 **Problem and product**
The gap in public NFL data, and the six screens that close it

03 **System architecture**
End-to-end flow, and the single decision the whole design follows from

04 **Technology stack**
Every dependency and the reason it is there

05 **The ETL pipeline**
nflverse → polars → 4,166 typed JSON files

06 **The data model and contract**
Eight file shapes, two absolute rules, one TypeScript source of truth

07 **Validation from both sides**
validate.py at build time, contract.ts at runtime

08 **Frontend architecture**
Routing, the data layer, the request cache, useAsync

09 **The replay engine**
The flagship: rAF, refs, useSyncExternalStore and a pure drawing module

10 **Feature walkthrough**
All six routes, screen by screen

11 **The six-season refactor**
Making every page mean one year, and keeping it across navigation

12 **Accessibility**
Measured contrast, roving tabindex, reduced motion, native semantics

13 **Performance**
Lighthouse, frame budget, bundle splitting, image sizing

14 **Testing strategy**
389 unit tests, 40 E2E specs, and 16 fault injections

15 **CI/CD and the weekly data refresh**
Scheduled ETL → validate → pull request → deploy

16 **Deployment and caching**
Vercel rewrites, two cache tiers, and the header bug production found

17 **War stories**
Fourteen real bugs, what caused them, and what each one taught

18 **Trade-offs and limitations**
What was given up, and what is knowingly left undone

19 **Interview Q&A**
Thirty likely questions with prepared answers

20 Appendices

File map, numbers cheat-sheet, commands, glossary

SECTION ONE

The 90-second pitch

If you get one question — “tell me about this project” — this is the answer.

> **THE OPENER**
>
> “Play-by-play NFL data is public and excellent — nflverse publishes every play of every game since 1999, win probability included — and almost entirely unreadable. It arrives as season-sized parquet files with hundreds of columns. That is perfect for a modelling notebook and useless if the question you actually have is *what did that comeback look like?*
>
> NFL Season Hub takes a visitor from all 32 teams down to a single play in three clicks, and then plays the game back to them: the home team’s win probability animates across the game one play at a time on an HTML canvas, at 60fps, with the decisive plays marked and a scrubber to drag. There is no backend. A Python ETL turns 273,623 plays into 4,166 typed static JSON files, and the browser reads them straight off a CDN.”

The three things worth steering the conversation toward

1. The replay engine

The one place the obvious React approach is wrong. A 60fps animation in `useState` is a render, reconciliation and commit per frame to move a dot a few pixels. Instead the play cursor is a `useRef`, a `requestAnimationFrame` loop repaints the canvas directly, and React learns only what a person can see — whether it is running, and which play it is on — through `useSyncExternalStore`. On a 218-play game that is eight renders a second instead of sixty.

2. The data contract, validated twice

`src/types/nfl.ts` is the single source of truth. `etl/validate.py` checks all 4,166 generated files against it from the producing side and is itself verified by 16 deliberate corruptions. `src/lib/contract.ts` checks the same contract at runtime in the browser, field by field, at the one place untrusted data enters the app. Past that module no file in the application needs a type assertion.

3. Everything is measured

Not “it feels fast”. Median frame 16.70ms across 240 consecutive frames of the longest game in the dataset, 0 dropped frames. Canvas drawing 0.14ms of a 16.7ms budget — which is why the obvious optimisation of caching the background to an offscreen bitmap was measured, found to cost more in blitting than it saved, and *not done*. Lighthouse run three times per route because one mobile run is not a measurement.

Scale, in one table

DIMENSION	FIGURE	NOTE
Seasons covered	2020–2026	Six complete, one in progress
Games with a replay	1,695	Regular season and postseason
Plays processed	273,623	From ~1.4M raw play-by-play rows
Generated JSON files	4,166	99 MB on disk, ~15 MiB packed in git
Player profiles	2,274	Of 3,137 rostered players
Average game file over the wire	6.6 KB	45.9 KB raw, gzipped by the CDN
Application source	~5,100 lines	TypeScript/TSX, excluding tests
Test code	~3,600 lines	20 Vitest files + 6 Playwright specs
ETL	1,913 lines	Python, stdlib + polars + nflreadpy

SECTION TWO

Problem and product

2.1 The problem

nflverse publishes every play of every NFL game since 1999, including the nflfastR win-probability model output. The data is free, high quality and well maintained. It is also shaped entirely for analysis, not for reading: season-sized parquet files with 300+ columns per play.

The gap is not the data. It is that nothing turns a row of `home_wp = 0.11` into the thing a person came for: the shape of a game.

A fan who remembers a comeback has no way to look at it. A sports-data engineer who wants to sanity-check a model has to load a notebook. Every existing consumer of this data is a programmer with a Python environment.

2.2 The product

A static site with six routes, designed as one drill-down spine: **League → Team → Game → Replay**, with two lateral branches (Compare, Players).

ROUTE	SCREEN	WHAT IT DOES
`/`	**League dashboard**	32 teams in any of six seasons, sortable by expected wins, record or division, filterable by conference and division. Season, sort and filters all live in the URL, so a 2021 dashboard sorted by record is a link you can send someone. Changing season costs zero bytes — it is a re-render.
`/team/:id`	**Team page**	One season, whole: that year's record against what the market expected of it, offensive and defensive splits on a printed scale, depth chart, roster, draft class, and every game as a route into its replay. Change the season and the entire page changes with it.
`/game/:id`	**Game replay**	The flagship. The home team's win probability animates across the game one play at a time on an HTML canvas, with ETL-flagged decisive plays marked along the timeline, a draggable scrubber, speed control, and a context box that always says where the game stands.
`/compare`	**Compare**	Any two teams on one shared scale, offense and defense, with every game they have played since 2020 and the series record — each meeting linking to its replay. This is the answer to the question the dashboard invites and cannot answer: is this team actually better than that one?
`/players`	**Player search**	2,274 profiles searchable by name, team or position, entirely in the browser against a 73 KB index. Accent- and punctuation-insensitive; ranked by surname prefix, then career games played.
`/player/:id`	**Player profile**	Bio, headshot, a season's stats, that season week by week with each game opening its replay, and a season-by-season career table spanning the whole dataset.

2.3 Product decisions worth defending

Every view is a URL
Sort, filter, conference, division, season, matchup and search query all serialise to the query string — and anything unrecognised falls back to a sensible default rather than erroring. A hand-edited or stale link degrades to a working dashboard, never to an empty grid or a dead end. Defaults are *omitted* from the URL, so the common case stays clean.

The key-play rule is decided once, in the ETL
A score, a turnover, or a win-probability swing of at least ten points. The UI reads the `isKeyPlay` flag and never applies a rule of its own, so the beads on the curve and the markers on the timeline can never come to mean different things.

Nothing is presented as something it is not

`meta.json` records how far through each season the data is. A season two games old gets replays but no team layer, no standings rows, and no place on the season control — and the league page says so above the controls, not in a footnote, because it changes how every number below it should be read.

Counts come from the file, never from prose

“Every player nflverse records production for — 2,274 of them” is computed at render time. The data refreshes weekly; a sentence with a number written into it goes stale silently.

SECTION THREE

System architecture

One decision drives the entire design: **the data never changes at runtime.** A finished season is a fact.



3.1 What that one decision buys

Because the data is historical and immutable, declaring it static removes, in a single move:

- the database
- the API layer
- cache invalidation
- cold-start latency
- connection pooling
- rate limiting
- auth for read paths
- every runtime failure mode more complex than a 404

What it replaces them with is a build step someone runs when a new week of football happens — which is now a scheduled GitHub Action that opens a pull request (Section 15).

3.2 The four layers

LAYER	LIVES IN	RESPONSIBILITY
Generation	`etl/build_data.py` `etl/refresh_rules.py`	Download raw nflverse frames, transform in polars, decide the key-play flag and the display season, emit exactly the contract's fields, write JSON strictly (`allow_nan=False`).
Verification	`etl/validate.py` `etl/test_refresh.py`	Nine required checks over every generated file, plus drift detection against `src/types/nfl.ts` itself. Fault-injected: 16 deliberate corruptions, 16 caught.
Access	`src/lib/data.ts` `src/lib/contract.ts` `src/lib/useAsync.ts`	The frontend's only door to the data. One promise cache keyed by URL, one runtime validator, one React bridge returning a discriminated union.
Presentation	`src/pages/*` `src/components/*` `src/lib/*` (pure)	Six lazily-loaded route chunks over one shared layout. All domain logic — sorting, comparison, search ranking, colour lifting, chart geometry — lives in pure modules under `lib/` so it can be argued with in a test rather than by squinting at a rendered page.

3.3 Directory map

```
nfl-season-hub/
├── etl/
│   ├── build_data.py       1,033 lines  the pipeline
│   ├── refresh_rules.py       99 lines  unattended decisions, dependency-free
│   ├── validate.py           648 lines  contract enforcement, stdlib only
│   ├── test_refresh.py       133 lines  fault injection
│   └── requirements.txt                 pinned exactly
├── public/data/                         4,166 generated files, committed
│   ├── meta.json                        freshness + season completeness
│   ├── teams-index.json                 32 teams, identity
│   ├── standings.json                   192 rows: every team, every season
│   ├── games-index.json                 1,695 games
│   ├── players-index.json               2,274 search entries
│   ├── team/<season>/<ID>.json          192 files
│   ├── player/<ID>.json                 2,274 files
│   └── game/<GAME_ID>.json              1,695 files
├── src/
│   ├── types/nfl.ts          392 lines  THE CONTRACT — single source of truth
│   ├── lib/                             pure logic + the data layer
│   ├── components/                      21 presentational components
│   ├── pages/                            6 lazily-loaded routes
│   ├── App.tsx                          the route table
│   └── main.tsx                         React root
├── e2e/                       6 specs   Playwright, desktop + Pixel 5
├── .github/workflows/
│   ├── ci.yml                           verify / data / e2e
│   └── refresh-data.yml                 Tuesday 09:00 UTC → PR
└── vercel.json                          rewrites + two cache tiers
```

3.4 Data flow for one visit

```
GET /game/2022_15_IND_MIN
  │
  ├─ Vercel rewrite: /((?!data/).*) → index.html          (SPA fallback,
  │                                                        excluding /data)
  ├─ entry chunk (84.78 kB gzip): React, Router, Layout
  ├─ React Router matches game/:id → lazy import GamePage  (6.29 kB gzip)
  │
  ├─ GamePage: useAsync(`game/${id}`, () => getGame(id))
  │     │
  │     └─ data.ts request()
  │           ├─ cache.get(url) → miss
  │           ├─ fetchJson(url)
  │           │     ├─ 404?           → DataError('not-found')
  │           │     ├─ !ok?           → DataError('network')
  │           │     ├─ content-type
  │           │     │  not json?      → DataError('not-found')   ← SPA fallback guard
  │           │     └─ response.json() throws? → DataError('malformed')
  │           ├─ cache.set(url, promise)     ← the PROMISE, not the value
  │           ├─ promise.catch(() => cache.delete(url))  ← failures retryable
  │           └─ parseGame(body, path)  ← every field checked; returns Game
  │                 └─ ContractError → DataError('malformed') naming the path
  │
  ├─ useAsync returns { status: 'success', data: Game }
  ├─ WinProbCanvas mounts → useReplay(game, legibleOn(CARD_SURFACE, homeColor))
  │     ├─ toChartPoints(game)        memoised
  │     ├─ quarterBoundaries(game)    memoised
  │     ├─ keyPlayIndices(game)       memoised
  │     ├─ ResizeObserver measures the canvas → first paint
  │     └─ requestAnimationFrame loop starts (unless prefers-reduced-motion)
  │
  └─ 60 frames/sec: cursor += elapsed × 8 × speed
                    → drawFrame(ctx, {...})           60×/s
                    → slider.value, --progress        60×/s (sub-pixel skipped)
                    → setPlayIndex(floor(cursor))      8×/s  → React re-render
```

SECTION FOUR

Technology stack

Nine runtime dependencies. Each one is here for a reason you can state in a sentence.

4.1 Runtime dependencies

PACKAGE	VERSION	WHY
`react` / `react-dom`	19.2	The component model, plus `useSyncExternalStore`, which is the supported way to read state that lives outside React — exactly what the animation loop is.
`react-router-dom`	7.18	Six routes and, importantly, `useSearchParams` — every view in this app is a URL.
`framer-motion`	13.2	Entrance and scroll animation on two pages only. 26 kB gzipped, so it is loaded with `LazyMotion` + the lightweight `m` components and confined to the League and Team chunks. Also supplies `useReducedMotion`.

4.2 Build and quality toolchain

TOOL	VERSION	ROLE AND REASONING
`vite`	8.2	Dev server and production bundler. Code-splitting per route is automatic from `React.lazy` + dynamic import.
`typescript`	6.0	Four project references: app, node (vite config), tests, e2e. `strict`, plus `noUncheckedIndexedAccess`, `noUnusedLocals`, `noUnusedParameters`, `erasableSyntaxOnly`, `noFallthroughCasesInSwitch`, `verbatimModuleSyntax`.
`tailwindcss` v4	4.3	Via `@tailwindcss/vite`. One custom theme token, `--color-muted: #858585`, because Tailwind's neutral-500 reaches only 4.04:1 on a card and neutral-600 just 2.46:1 — both short of the 4.5:1 normal text owes.
`vitest`	5.0	389 unit and component tests in jsdom. Shares Vite's config and the `@/` alias, so there is no second module-resolution story.
`@testing-library/react`	16.3	Component tests driven through the accessible tree, not through implementation details.
`@playwright/test`	1.63	40 E2E specs across two device profiles. Runs against `vite preview` (the production build), not the dev server — see 4.4.
`oxlint`	1.79	What `create-vite` scaffolds now; same role as ESLint, considerably faster. Enforces `react/rules-of-hooks` as an error.
`prettier`	3.9	Format check is a CI gate, not a suggestion.

4.3 The Python side

PACKAGE	PIN	REASONING
`nflreadpy`	==0.1.5	nflverse's supported Python loader. **Pinned exactly**: it is pre-1.0 and its loaders have already changed shape once (the 2025 depth-chart schema), so a floating range would make the pipeline silently non-reproducible.
`polars`	==1.44.2	`nflreadpy` returns polars frames natively.
`pyarrow`	==25.0.1	Parquet backing for the above.
`pandas`	**Deliberately absent.** Every transform here is polars; adding pandas would mean converting frames back and forth for no benefit. Nothing imports it. (The original project brief specified pandas — this is a documented deviation, not an oversight.)	

> **A STACK QUESTION YOU WILL PROBABLY GET**
>
> **"Why `nflreadpy` and not `nfl_data_py`?"** — `nfl_data_py` is the better-known package and it is what most tutorials name. It is also deprecated upstream and pins `pandas<2` / `numpy<2`, neither of which has a Python 3.12 wheel. It will not install on a current interpreter. `nflreadpy` is nflverse's own successor and every function has an equivalent (`import_pbp_data` → `load_pbp`, and so on).

4.4 Two configuration choices that encode real bugs

Playwright runs against `vite preview`, not `vite dev`

The two differ in a way this app depends on. `vite preview` answers a missing data file with `index.html` and a **200** — which is exactly the case `src/lib/data.ts` guards against with its content-type check, and which the dev server does not reproduce. Testing against the dev server would leave that guard permanently unexercised.

The e2e project list is desktop *and* Pixel 5

The replay is the feature most likely to break on a small screen: the scrubber, the key-play rail and the transport all share one row. Both device profiles run every spec.

```
projects: [
  { name: 'desktop', use: { ...devices['Desktop Chrome'] } },
  { name: 'phone',   use: { ...devices['Pixel 5'] } },
],
webServer: {
  command: 'npm run build && npm run preview -- --port 4173',
  url: 'http://localhost:4173',
  timeout: 180_000,
},
```

4.5 The verify gate

```
npm run verify
  = typecheck  (tsc -b, four projects)
  → lint       (oxlint)
  → format:check
  → test       (389 Vitest tests)
  → build      (tsc -b && vite build)
```

This is exactly what CI runs, in the same order, so a green local run means a green CI run.

SECTION FIVE

The ETL pipeline

`etl/build_data.py`, 1,033 lines, turns roughly 1.4 million raw play-by-play rows into 4,166 files that average 6.6 KB over the wire.

5.1 Inputs

NFLREADPY LOADER	WHAT IT SUPPLIES
`load_schedules()`	Every fixture, its result, and the closing point spread. ~7,500 rows. Loaded once for all seasons — asking for a range past what nflverse publishes is an error, not an empty frame, so a new season is *discovered* by filtering afterwards.
`load_pbp(seasons=[s])`	Play-by-play, ~300 columns. The most expensive call; loaded once per season and reused for both the game layer and the team stat lines.
`load_teams()`	Identity: name, conference, division, colours, ESPN logo URL.
`load_rosters(seasons=[s])`	Full season roster including players later cut, so a team carries 95–110 names rather than a 53-man active list.
`load_depth_charts(seasons=[s])`	Depth order. Two incompatible schemas — see 5.5.
`load_draft_picks(seasons=[s])`	Draft class, keyed by Pro-Football-Reference abbreviations.
`load_player_stats(seasons=[...])`	Weekly per-player statistics, ~150 columns, filtered to `season_type == "REG"`.

5.2 Every threshold is a named constant at the top of the file

```
SEASONS = [2020, 2021, 2022, 2023, 2024, 2025]

# Standard deviation of NFL game margins. Converting a closing point spread
# to a win probability with the normal CDF at this sigma is the conventional
# approximation, and it is what `projectedWins` is built from.
MARGIN_SIGMA = 13.86

KEY_PLAY_WP_SWING = 0.10     # a ten-point win-probability move

REPLAY_PLAY_TYPES = {"pass", "run", "punt", "field_goal", "kickoff",
                     "extra_point", "qb_kneel", "qb_spike", "no_play"}

MAX_DESCRIPTION = 140        # full nflverse descriptions run to ~250 chars
                             # and are the single largest contributor to
                             # game file size

ABBR_FIXES = {  # relocations
                "OAK": "LV", "SD": "LAC", "SDG": "LAC", "STL": "LA", "LAR": "LA",
                # Pro-Football-Reference codes, used by the draft dataset
                "GNB": "GB", "KAN": "KC", "LVR": "LV", "NWE": "NE",
                "NOR": "NO", "SFO": "SF", "TAM": "TB" }

PLAYER_OPTIONAL = ("number", "age", "college", "status")
PLAY_OPTIONAL   = ("down", "distance", "epa")

PROMOTE_MIN_PLAYED = 64      # in refresh_rules.py — see Section 15
```

This matters in an interview: when someone asks “what counts as a key play?” the answer is a constant with a comment above it, not a number buried in a conditional.

5.3 The key-play rule

`isKeyPlay` is true when **any** of these hold:

1. the home win probability moved at least `KEY_PLAY_WP_SWING` (0.10) from the previous play
2. the play scored a touchdown
3. the play was a turnover (interception or lost fumble)
4. the play was a made field goal

Rule 1 catches drama the box score misses — a stop on 4th and goal. Rules 2–4 guarantee every score and giveaway is marked even in a blowout where the needle no longer moves. Together this flags about **9.5% of plays**, averaging 15 per game, with a maximum of 41.

```
is_key = (
    swing >= KEY_PLAY_WP_SWING
    or bool(r.get("touchdown"))
    or bool(r.get("interception"))
    or bool(r.get("fumble_lost"))
    or bool(r.get("field_goal_result") == "made")
)
```

5.4 Building one game's plays — three defects fixed in nine lines

```
rows = (
    game_pbp.filter(
        pl.col("home_wp").is_not_null()
        & pl.col("play_type").is_in(sorted(REPLAY_PLAY_TYPES))
        # (a) Timeouts are logged as no_play rows and often carry a stale
        # score (frequently 0–0), which would make the scoreboard jump.
        & (pl.col("timeout").fill_null(0) != 1)
    )
    # (b) play_id is NOT reliably chronological: penalty and replay-review
    # rows can carry a later id than the snap they belong to. The game clock
    # is the real ordering.
    .sort(["qtr", "game_seconds_remaining", "play_id"],
          descending=[False, True, False])
    .iter_rows(named=True)
)

for r in rows:
    wp = min(max(num(r["home_wp"], 3), 0.0), 1.0)   # (c) guarantee 0..1
    # Football scores only ever increase; a running max repairs any row
    # whose score field lags the play it belongs to.
    max_home = max(max_home, whole(r["total_home_score"]) or 0)
    max_away = max(max_away, whole(r["total_away_score"]) or 0)
```

DEFECT	SYMPTOM IT CAUSED
(a) Timeout rows	~2,200 rows per season logged as `no_play` carrying a stale score, frequently 0–0. The replay scoreboard reset mid-game.
(b) Non-chronological `play_id`	Penalty and replay-review rows sort after the snap they belong to. The replay scores jumped backwards.
(c) Unclamped probability	The model can emit a hair outside 0..1. The contract promises 0..1, so it is clamped at the producer rather than defended at every consumer.

5.5 The 2025 depth-chart schema change

nflverse changed this dataset's shape mid-dataset. Both schemas are handled and normalise to the same output:

SEASONS	SHAPE	COLUMNS
≤ 2024	Weekly snapshots	`club_code`, `depth_team`, `week`, `football_name`, `last_name`
≥ 2025	Dated snapshots	`team`, `pos_abb`, `pos_rank`, `player_name`, `dt`

Two sub-traps inside this:

- **For legacy seasons, the *last regular-season week* is used, not the last week.** Later weeks exist but cover only teams still in the playoffs — taking the maximum week left 30 of 32 teams with no depth chart at all.
- **`football_name` is the *first* name**, not the full name. It has to be joined with `last_name`.

5.6 Expected wins, and why it is not a Vegas over/under

The nflverse Vegas win-totals dataset (`nfldata/win_totals.csv`) was discontinued after 2020. Rather than ship one season of a real over/under and five of nothing, `projectedWins` is **market-implied expected wins**: each game's closing spread converted to a win probability and summed across the regular season.

```
def norm_cdf(x: float) -> float:
    return 0.5 * (1.0 + math.erf(x / math.sqrt(2.0)))

def win_prob_from_spread(spread: float) -> float:
    """Closing spread (positive = home favored) -> P(home win)."""
    return norm_cdf(spread / MARGIN_SIGMA)

# summed across the regular season, both sides of every game:
p_home = win_prob_from_spread(spread)
bucket[home]["expectedWins"] += p_home
bucket[away]["expectedWins"] += 1.0 - p_home
```

The sign convention (positive `spread_line` = home favoured) was verified empirically against known results rather than assumed. **The dashboard says all of this in a footnote on the page**, because presenting a derived figure under a name that implies a Vegas line would be misleading.

Read against the record beside it, the gap is an over/underperformance figure: the 2021 Chiefs won 12 games the market priced at 10.4.

5.7 Missing data: the two rules

> **RULE ONE — ABSENT MEANS ABSENT**
> An unknown value is **omitted** from the JSON, never emitted as `null`. `null` does not satisfy `number | undefined` under `strict`, so omission is the only representation that typechecks — and it means "no down on this play" (a kickoff) never renders as a placeholder pretending to be data. 2,276 key plays across six seasons have no down.

CASE	TREATMENT
Optional contract fields	Key removed entirely by `drop_absent()`.
`distance` without `down`	Emitted only alongside a down. Kickoffs and extra points carry `ydstogo = 0` upstream, which would produce a meaningless `distance: 0`.
`-0.0`	Collapsed to `0.0`.
Required string fields	Fall back to `""`, never `null` — a few draft picks have no listed position or college.
Required stat fields	Fall back to `0` when a team has no qualifying plays.
`NaN` / `Infinity`	Converted to `None` on the way out, and `json.dumps(allow_nan=False)` makes any survivor a **hard build error** rather than invalid JSON that a browser would reject at parse time.

`homeWinProb`	Clamped to 0..1 at the producer.
`age`	Computed against September 1 of the season, and rejected outside 15–60.
Player stat zeros	Dropped. A quarterback's file carries no tackle count and a cornerback's no completions — which is *why* the UI asks which groups have numbers rather than branching on position.
`fg_long` in a season total	Taken as a **maximum**, not summed. Summing it would invent a 300-yard field goal.

5.8 Regular season only, and why

Records, per-game rates and expected wins are all regular-season figures so they stay comparable across teams. Postseason games still get replays — they are just excluded from the rate denominators.

This was found the hard way: **including playoff plays had inflated KC's 2023 offense to 444 yards per game against an official 344.** A team that plays four extra games accumulates yards against a 17-game denominator.

5.9 The player layer

2,274 files from 3,137 rostered players. The 863 without one have no stat row in any season — 346 of the 550 offensive linemen among them, because nflverse records a statistic for a lineman only when he catches a pass or recovers a fumble.

> **A DETAIL WORTH QUOTING**
>
> The remaining 204 linemen *do* have pages, which is why the code says "players with production" and never "skip the linemen". And which numbers a page shows is decided by which numbers *exist*, never by the position in the file — Mahomes' 2025 line contains one tackle and one reception, and a page that branched on "quarterback" would have hidden both, and would have been wrong for every two-way player in the league.

Every season's roster carries `hasProfile`, so the UI links a name only when there is a page behind it. A 2020 roster is full of players who have since retired: they are named and not linked, the same rule the linemen fall under.

5.10 Dataset quirks that cost real time to discover

QUIRK	CONSEQUENCE IF MISSED
The draft dataset uses Pro-Football-Reference abbreviations (`GNB`, `KAN`, `LVR`, `NWE`, `NOR`, `SFO`, `TAM`, `LAR`)	8 teams per season had an empty draft class. Normalised through `ABBR_FIXES`, which also covers relocations (OAK→LV, SD→LAC, STL→LA).
2020 had 16-game regular seasons; BUF and CIN played 16 in 2022 (the cancelled game)	Any hard-coded 17 produces wrong per-game rates.
Postseason overtime is 15 minutes; regular-season overtime is 10	A fixed 600 computes an elapsed time *earlier* than the end of regulation for a playoff OT game. This is why `GamePlay` carries no whole-game time and `elapsedSeconds()` takes `gameType`.
`load_rosters` returns the full season roster, including `status: "CUT"`	A team carries 95–110 players, not 53. The UI defaults to filtering on `status === 'ACT'`.
Headshots are 3–6 MB PNGs on the NFL's Cloudinary CDN	See Section 13.5 — this is one of the best stories in the project.

SECTION SIX

The data model and contract

Eight file shapes, mirrored exactly by `src/types/nfl.ts`. That file is the single source of truth, and three things must change together: the interface, the ETL emitter, and the validator schema.

6.1 The eight shapes

FILE	COUNT	RAW / GZIP	CONTENTS
`meta.json`	1	0.6 / 0.2 KB	When the data was generated, how complete each season is, which seasons have a team layer
`teams-index.json`	1	8 / 1.4 KB	32 teams: identity, colours, last season, projection
`standings.json`	1	21 / **2.6 KB**	192 rows: every team's record and expected wins, every season
`games-index.json`	1	239 / 26 KB	1,695 games: ids, dates, scores, type
`players-index.json`	1	394 / 73 KB	Search index: name, position, team, headshot, career games
`team/<season>/<ID>.json`	192	23 / 3.8 KB	One team in one season: record, roster, depth chart, draft class, splits, games
`player/<ID>.json`	2,274	4.3 / 0.9 KB	Bio, headshot, career seasons, every season week by week
`game/<GAME_ID>.json`	1,695	45.9 / **6.6 KB**	Every play: clock, score, down, description, win probability

A game averages 6.6 KB over the wire, which is why a replay can load on demand with no backend.

6.2 The core interfaces

```
export interface GamePlay {
  playId: number
  quarter: number
  /**
   * Seconds remaining in `quarter`, counting down. 900 at the start of a
   * regulation quarter; 0 at its end. Overtime length depends on the game:
   * 600 in the regular season, 900 in the postseason. Use overtimeLength()
   * — do NOT assume 600.
   */
  clockSeconds: number
  /**
   * 0..1 — the home team's win probability BEFORE this play is run
   * (pre-snap). The replay animates this value, so the curve shows the state
   * going INTO each play; the final play therefore need not sit at exactly
   * 0 or 1 when a game ends on a walk-off score.
   */
  homeWinProb: number
  scoreHome: number
  scoreAway: number
  down?: number
  distance?: number
  posteam: string
  playType: string
  description: string
  epa?: number
  /** See KEY_PLAY rules in etl/build_data.py: score, turnover, or big swing. */
  isKeyPlay: boolean
}

export interface Game {
  gameId: string
  season: number
  week: number
  date: string
  /** Needed to interpret overtime length. */
  gameType: GameType          // 'REG' | 'WC' | 'DIV' | 'CON' | 'SB'
  home: GameTeam
  away: GameTeam
  /** Chronological: quarter ascending, then game clock descending. */
  plays: GamePlay[]
}
```

6.3 What `clockSeconds` means — and what it deliberately does not store

Seconds remaining in `quarter`, counting down. 900 at the start of a regulation quarter, 600 at the start of regular-season overtime, 0 at its end. It is *not* elapsed time and *not* whole-game time.

Whole-game elapsed time is deliberately not stored, to keep game files to the contract's fields only — at 1,695 files, one extra integer per play is real bytes. It is derived on the client instead:

```
export function overtimeLength(gameType: GameType): number {
  return gameType === 'REG' ? 600 : 900
}

export function elapsedSeconds(play: GamePlay, gameType: GameType): number {
  if (play.quarter <= 4) {
    return (play.quarter - 1) * 900 + (900 - play.clockSeconds)
  }
  const ot = overtimeLength(gameType)
  return 3600 + (play.quarter - 5) * ot + (ot - play.clockSeconds)
}
```

6.4 The shape decisions worth explaining

`TeamSeason` is deliberately *not* `TeamSummary` with extra fields

It would be tempting to extend. `TeamSummary` carries `lastSeason` and `projectedWins`, both of which are relative to *now*. A season page wants *that season's* record and what the market expected of it. Same numbers, different question — so `TeamSeason` is its own contract and the index is untouched.

`standings.json` is a new file rather than three more fields on `teams-index.json`

The dashboard's whole appeal is that sorting and filtering never touch the network, and a season switch had to work the same way — which means holding every season at once, 2.6 KB for all 192 rows. Putting that on `teams-index.json` would have charged the comparison page and every team page for it too.

`TeamSeasonView` is a shape no file has

Built in the browser by joining `teams-index.json` (identity: colours, division — what a team *is now*) to `standings.json` (a record, which belongs to a season). A team with no row for that season is **dropped rather than shown at 0-0** — that is a relocation or an expansion, not a winless year.

6.5 The second absolute rule

> **RULE TWO — `ISKEYPLAY` IS DECIDED ONCE, IN THE ETL**
>
> A score, a turnover, or a win-probability swing of at least ten points. The UI reads that flag and never applies a rule of its own, so the beads on the curve and the markers on the timeline can never come to mean different things. `keyPlayIndices(game)` in `lib/winprob.ts` is four lines and contains no threshold.

SECTION SEVEN

Validation from both sides

The same contract is enforced twice: at build time from the producing side, and at runtime from the consuming side. Neither is redundant — they catch different failures.

7.1 `etl/validate.py` — the producing side

648 lines, **standard library only**, so CI can run it against the committed JSON with a bare Python and no ETL run, no install, and no nflverse download. It covers nine required checks:

#	CHECK
1	Exactly 32 teams exist
2	Every required field is present, with the right runtime type
3	No duplicate team ids
4	Game ids are well-formed (`^\d{4}_\d{2}_[A-Z]{2,3}_[A-Z]{2,3}$`) and unique
5	`homeWinProb` is a real number in 0..1
6	Plays are chronologically ordered — quarter ascending, clock descending
7	Every team game reference resolves to a generated game file
8	Output re-serialises cleanly (no NaN, Infinity or non-JSON objects)
9	Optional (`?`) properties are omitted rather than emitted as `null`

Plus a set of domain invariants beyond the required nine:

- Colours match `#rrggbb` ; `projectedWins` sits inside a plausible 0..25
- Overtime clock must not exceed the game type's period length (600 REG / 900 postseason)
- `posteam` on every play must be one of the two teams in *that* game
- Scores are monotonic and the replay ends on the real final score
- Season directories must be exactly the ones `meta.teamSeasons` advertises — a directory nothing links to is dead weight still served to anyone with the URL, and a missing one is a season the dashboard offers and cannot open
- Every stored headshot must match the exact Cloudinary transformation `headshotAt()` knows how to resize — **a new delivery type has to fail here rather than on someone's phone**
- The search index and the pages behind it must be the same set, or the search offers results that 404 or hides pages nothing links to

The check that validates the validator: contract drift

The Python schemas are a hand-written mirror of the TypeScript interfaces. **If the two drift apart, every other check silently validates the wrong shape** — a validator that passes against a contract nobody holds is worse than no validator. So the agreement is itself verified by parsing `src/types/nfl.ts` :

```
def ts_fields(name: str):
    # The negative lookahead matters: "Game" would otherwise prefix-match
    # "GameSummary" and compare the wrong interface.
    m = re.search(rf"export interface {name}(?![A-Za-z0-9_])[^{{]*\{{(.*?)\n\}}",
                  ts, re.S)
    ...
    depth = 0
    for line in m.group(1).splitlines():
        line = re.sub(r"//.*", "", line).strip()
        # Only fields of the interface itself. Without the depth count,
        # `stats: { offense: ...; defense: ... }` reported offense and
        # defense as top-level fields the validator was missing.
        if depth == 0:
            f = re.match(r"(\w+)(\??):", line)
            if f: found[f.group(1)] = f.group(2) == ""
        depth += line.count("{") - line.count("}")
```

It reports three distinct drift kinds: a field the validator has and `nfl.ts` does not; a field `nfl.ts` has and the validator does not; and a field whose *optionality* disagrees.

Four real defects it caught

1. Non-chronological `play_id`
2. Timeout rows carrying stale scores
3. Playoff games inflating per-game rates
4. The 2025 depth-chart schema change upstream

The validator is itself fault-injected: 16 corruptions, 16 caught

1. missing team
2. duplicate team id
3. missing required field
4. null in an optional slot
5. malformed game id
6. duplicate game id
7. out-of-range win probability
8. shuffled play order
9. dangling game reference
10. literal `NaN`
11. wrong runtime type
12. unexpected field
13. `distance` without `down`
14. a `posteam` not in the game
15. overtime clock exceeding its game type
16. a missing `gameType`

7.2 `src/lib/contract.ts` — the consuming side

`response.json()` hands back an `unknown`. The usual `as Game` turns a stale deploy, a half-written ETL run, or an SPA fallback serving HTML into a crash somewhere far away from the fetch that caused it.

So every field is checked at the one place untrusted data enters the app, and a failure reads:

```
/data/game/2023_12_NO_ATL.json does not match the data contract —
game.plays[0].homeWinProb: expected a finite number, received "0.5"
```

The combinator design

```
type Parser<T> = (value: unknown, path: string) => T

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value)
}

function number(value: unknown, path: string): number {
  // NaN and Infinity are rejected rather than passed on: JSON cannot carry
  // either, so their presence means the value was computed wrong upstream,
  // and a chart silently plots NaN as a gap.
  if (typeof value !== 'number' || !Number.isFinite(value)) {
    throw new ContractError(path, 'a finite number', value)
  }
  return value
}

function optional<T>(parse: Parser<T>): Parser<T | undefined> {
  // `undefined` only, never `null`: accepting null here would quietly let
  // the ETL emit a second spelling of "missing" that no consumer expects.
  return (value, path) => (value === undefined ? undefined : parse(value, path))
}

function field<T>(source, key: string, parse: Parser<T>, path: string): T {
  return parse(source[key], `${path}.${key}`)   // path extends as it descends
}
```

Three properties of this module worth naming

Adding a contract field is a compile error until it is checked

The parsers return the interfaces from `types/nfl.ts` **by annotation**, not by assertion. Add a property to an interface and the parser that builds it no longer satisfies its return type.

No type assertion exists anywhere else in the application

Past this module the types are *earned* rather than asserted. `isRecord` is a type predicate, not a cast — reading an unknown key off any non-null, non-array object genuinely yields `unknown`, so the narrowing is sound where `value as Record<string, unknown>` would only have been claimed. `oneOf` uses `find` rather than `includes`, because `find` returns the member itself, already typed.

Unknown extra properties are ignored on purpose

The ETL is allowed to emit a field this build has not learned about yet. Tolerating extras is what makes the data layer and the app independently deployable.

What it costs — measured, not assumed

MEASUREMENT	FIGURE
Validating the largest file the app loads (239 KB, 1,695-game index)	0.23 ms
`JSON.parse` on the same file, for comparison	0.57 ms
Added to the shared data chunk (1.85 → 3.02 kB), gzipped	1.17 kB

Because validation is that cheap, the cache stores the unchecked JSON and re-validates on each read, rather than storing a typed value the cache would have to assert the type of. Re-checking is cheaper than the assertion would have been worth.

SECTION EIGHT

Frontend architecture

8.1 Routing

Six routes inside one shared layout, each page a lazy chunk:

```
const LeaguePage  = lazy(() => import('@/pages/LeaguePage'))
const TeamPage    = lazy(() => import('@/pages/TeamPage'))
const GamePage    = lazy(() => import('@/pages/GamePage'))
const ComparePage = lazy(() => import('@/pages/ComparePage'))
const PlayerPage  = lazy(() => import('@/pages/PlayerPage'))
const PlayersPage = lazy(() => import('@/pages/PlayersPage'))

<Routes>
  <Route element={<Layout />}>
    <Route index          element={<LeaguePage />} />
    <Route path="team/:id" element={<TeamPage />} />
    <Route path="game/:id" element={<GamePage />} />
    <Route path="compare"  element={<ComparePage />} />
    <Route path="players"  element={<PlayersPage />} />
    <Route path="player/:id" element={<PlayerPage />} />
    <Route path="*"        element={<NotFound />} />
  </Route>
</Routes>
```

The payoff is real and measured: **a league visitor never downloads the replay engine, and a comparison downloads neither the replay engine nor the motion runtime.**

The Suspense fallback is route-shaped, not a spinner

Without this the shell showed a spinner while the chunk arrived, and the page then showed its own skeleton while the data arrived — **two different waiting states in a row for one click.** The path is enough to know which shape is coming:

```
function RouteSkeleton() {
  const { pathname, search } = useLocation()
  if (pathname.startsWith('/game/'))    return <GameSkeleton />
  if (pathname.startsWith('/team/'))    return <TeamSkeleton />
  if (pathname.startsWith('/players'))  return <PlayersSkeleton />
  if (pathname.startsWith('/player/'))  return <PlayerSkeleton />
  if (pathname.startsWith('/compare')) {
    // A link with both teams in it is going to render a full comparison,
    // so reserve that shape rather than the pickers alone.
    const params = new URLSearchParams(search)
    return <CompareSkeleton matchup={params.has('a') && params.has('b')} />
  }
  return <LeagueSkeleton />
}
```

8.2 The data layer: `src/lib/data.ts`

Components never call `fetch`, never build a URL, and never see an untyped value. Eight typed accessors and one cache.

```
const DATA_ROOT = `${import.meta.env.BASE_URL.replace(/\/$/, '')}/data`

export type DataErrorKind = 'not-found' | 'network' | 'malformed'

export class DataError extends Error {
  readonly kind: DataErrorKind
  readonly url: string
  readonly status?: number
}
```

Three error kinds, because those are the three things a UI can meaningfully branch on: “that does not exist” (offer a way back), “the network failed” (offer a retry), and “the data is wrong” (a bug, report it clearly).

The request cache stores the *promise*, not the value

```
const cache = new Map<string, Promise<unknown>>()

async function request<T>(path: string, parse: Parser<T>): Promise<T> {
  const url = `${DATA_ROOT}/${path}`
  let pending = cache.get(url)
  if (!pending) {
    pending = fetchJson(url)
    cache.set(url, pending)
    pending.catch(() => cache.delete(url))   // let failures be retried
  }
  const body = await pending
  try {
    return parse(body, path)
  } catch (cause) {
    if (cause instanceof ContractError) {
      throw new DataError('malformed', url,
        `${url} does not match the data contract — ${cause.message}`)
    }
    throw cause
  }
}
```

Why the promise	Two components asking for the same file during the same tick share *one* network request. The team page fetches `teams-index.json` and so does the footer; only one goes out.
Why evict rejections	A cached rejected promise would replay the same failure forever. Evicting on rejection means the retry button issues a genuinely new request.
Why `unknown` and not the typed value	Handing a cached `Promise<unknown>` back as a `Promise<Game>` would need an assertion on every hit — the cache cannot know which parser filled the entry. Re-validating costs 0.23 ms on the largest file; the assertion would have cost the project’s central invariant.

Four guards in `fetchJson`, in order

```
try { response = await fetch(url) }
catch { throw new DataError('network', url, `Could not reach ${url}.`) }

if (response.status === 404) throw new DataError('not-found', url, ..., 404)
if (!response.ok)            throw new DataError('network',   url, ..., response.status)

// A misconfigured SPA fallback answers a missing file with index.html and
// a 200. Catching it here turns a confusing parse error into 'not-found'.
const contentType = response.headers.get('content-type') ?? ''
if (!contentType.includes('json')) {
  throw new DataError('not-found', url,
    `Expected JSON at ${url} but the server returned "${contentType}".`)
}

try { return await response.json() }
catch { throw new DataError('malformed', url, `The data at ${url} is not valid JSON.`) }
```

WHY THE CONTENT-TYPE GUARD EXISTS TWICE OVER

`vercel.json` excludes `/data` from the SPA history fallback, so production returns a real 404. But `vite preview` — which the entire E2E suite runs against — answers a missing data file with `index.html` and a 200. The two environments differ, so **both guards are needed**, and an E2E test requests a game that is not in the dataset specifically to prove the client-side one works.

8.3 The React bridge: `useAsync`

```
export type AsyncState<T> =
  | { status: 'loading' }
  | { status: 'error'; error: DataError; retry: () => void }
  | { status: 'success'; data: T }

export function useAsync<T>(key: string, load: () => Promise<T>): AsyncState<T> {
  const [settled, setSettled] = useState<Settled<T> | null>(null)
  const [attempt, setAttempt] = useState(0)
  const retry = useCallback(() => setAttempt((n) => n + 1), [])

  // `load` is a fresh closure every render, but only `key` should trigger a
  // refetch. Declared first so the ref is current before the fetch reads it.
  const latestLoad = useRef(load)
  useEffect(() => { latestLoad.current = load })

  useEffect(() => {
    let active = true
    latestLoad.current()
      .then((data) => { if (active) setSettled({ key, state: { status: 'success', data } }) })
      .catch((cause) => { if (active) setSettled({ key, state: { status: 'error', ... } }) })
    return () => { active = false }
  }, [key, attempt, retry])

  return settled?.key === key ? settled.state : { status: 'loading' }
}
```

Three design points

A discriminated union, so a component cannot read `data` without proving it succeeded

There is no `data: T | undefined` anywhere. Every page reads `if (state.status === 'loading') return <Skeleton />`, then `if (state.status === 'error') return <ErrorState />`, and only then does `state.data` exist to TypeScript.

Keyed on a string, not a dependency array

The key identifies the resource, mirroring the URL key the data cache uses. It keeps the effect statically analysable and makes an accidental refetch loop impossible — the single most common bug in hand-rolled fetch hooks.

Loading is *derived*, not stored
A result tagged with a stale key simply reads as loading. This avoids a redundant render on every key change, **and it is what stops a slow request for one team from painting over a fast one for the next.** There is no race to lose, because the answer to "whose result is this?" is in the result.

8.4 Pure logic modules

All domain logic lives under `src/lib/` as pure functions, kept out of components so the rules are readable in one place and testable without rendering.

MODULE	LINES	RESPONSIBILITY
`league.ts`	257	Sorting, filtering, the season join, URL (de)serialisation, and the live-region sentence. Every comparator ends in a name tiebreak, so order is fully determined rather than relying on input order.
`season.ts`	128	Reading `meta.json` in words: `refreshedAgo`, `describeCoverage`, `needsCoverageNotice`, `resolveSeason`.
`football.ts`	267	Domain presentation: down & distance, week labels, position groups, depth-chart units, roster status codes, `orDash` (never emits "undefined", "null" or "NaN").
`colors.ts`	118	WCAG luminance and contrast, `readableTextOn`, `legibleOn`, `teamAccent`, `withAlpha`. See Section 12.
`winprob.ts`	76	Shaping plays into a chart series, quarter boundaries, key-play indices, clock labels.
`replayCanvas.ts`	414	The pure drawing module. See Section 9.
`useReplay.ts`	411	The animation clock. See Section 9.
`compare.ts`	103	`leads` (with `lowerIsBetter`), `meetings`, `seriesRecord`, matchup URL state.
`playerSearch.ts`	111	Normalisation and four-tier ranking. See 10.5.
`playerStats.ts`	132	Five stat groups, and `primaryGroup` — what a player is mainly measured by, derived from what they actually did.
`logos.ts`	73	`logoAt` and `headshotAt` — requesting images at the size they are drawn.
`stats.ts`	38	The five team metrics and the fixed scale each is read against, shared by the team page and the comparison so the same team never looks different on two screens.

A representative example: the comparison leader

```
export function leads(a: number, b: number, lowerIsBetter = false): Side {
  if (!Number.isFinite(a) || !Number.isFinite(b) || a === b) return null
  const aWins = lowerIsBetter ? a < b : a > b
  return aWins ? 'a' : 'b'
}
```

`lowerIsBetter` is the whole reason this is a function rather than a `>`. Every defensive metric is a quantity *conceded*, so the same five numbers that mean "good" on offense mean the opposite one panel over, and a comparison that got that backwards would confidently award the wrong team. An exact tie returns `null`: marking one of two identical numbers as the leader would be inventing a distinction the data does not make.

SECTION NINE

The replay engine

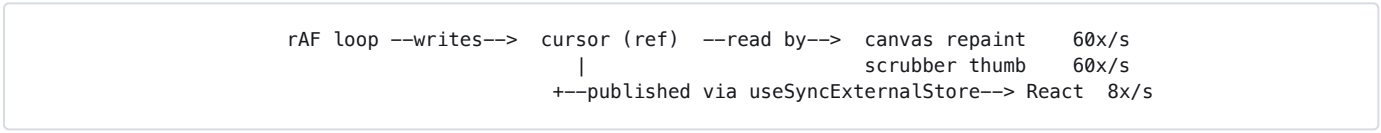
This is the reason the project exists, and the one place where the obvious React approach is the wrong one. Expect most of the deep technical questioning to land here.

9.1 The argument

> **WHY NOT REACT STATE**
>
> The animation advances a cursor 60 times a second. Storing that cursor in `useState` means a render, a reconciliation and a commit *per frame* — for a chart whose only actual change is a few hundred pixels of line. That is a lot of machinery to move a dot.

What it does instead. The cursor is a `useRef`: a fractional play index. A `requestAnimationFrame` loop advances it by elapsed time and repaints the canvas directly. One frame costs one canvas repaint and nothing else.

How React finds out. The two facts a person can actually see — whether it is running, and which play it is on — are published through `useSyncExternalStore`, which is React's supported way to read state that lives outside React. It re-renders when the *value* changes: once per play, not once per frame. **On a 218-play game at 1×, that is eight renders a second instead of sixty.**



9.2 The division of labour

OWNER	OWNS
React	The shell, the loaded game, the accent colour, and the transport state a person can see and click.
The loop	The cursor, the previous frame's timestamp, and the pending frame handle. All three live in refs. **None of them causes a render.**

There is exactly **one** playback position: `cursor`, a fractional play index. The canvas, the scrubber thumb, the readout and every transport action read and write that one ref. Nothing derives position from a frame count.

9.3 `useLoopValue` — the bridge

```
/**
 * A value the animation loop writes and React reads.
 *
 * The loop is an external system — it runs on the browser's frame clock, not
 * on React's — so its visible state is exposed the way React asks external
 * state to be exposed, rather than mirrored into `useState`. The write is a
 * plain assignment plus a notification: safe to call from inside a frame
 * callback, and silent when the value has not actually changed.
 */
function useLoopValue<T>(initial: T): [T, (next: T) => void] {
  const value = useRef(initial)
  const listeners = useRef(new Set<() => void>())

  const subscribe = useCallback((listener: () => void) => {
    listeners.current.add(listener)
    return () => { listeners.current.delete(listener) }
  }, [])

  const read = useCallback(() => value.current, [])

  const write = useCallback((next: T) => {
    if (Object.is(next, value.current)) return        // silent when unchanged
    value.current = next
    for (const listener of listeners.current) listener()
  }, [])

  return [useSyncExternalStore(subscribe, read), write]
}
```

Three values use it: `playing`, `playIndex`, and `hoverIndex`. The `Object.is` guard is what turns 60 writes per second into 8 renders per second — `setPlayIndex(Math.floor(next))` is called every frame and does nothing 52 times out of 60.

> **THE QUESTION BEHIND THE CHOICE**
>
> **"Why `useSyncExternalStore` rather than a mutable ref plus a forced re-render?"** — The loop genuinely *is* an external system on a different clock. Using React's own escape hatch for that keeps it correct under concurrent rendering, instead of relying on the render phase observing a mutation it is not supposed to see. A `forceUpdate` pattern happens to work today and is not guaranteed to under concurrent features; this is guaranteed.

9.4 The frame step

```
const step = useCallback(
  // A named function expression, so the body can schedule the next frame with
  // itself: `advance` is bound inside its own scope, where the `const` holding
  // an arrow function would not yet be initialized.
  function advance(now: number) {
    const previous = previousFrameTime.current
    previousFrameTime.current = now

    // Clamped at BOTH ends. A frame's timestamp can predate the one that
    // scheduled it, which would run the replay backwards, and a tab returning
    // from the background reports an arbitrarily large gap.
    const delta = previous === null
      ? 0
      : Math.min(Math.max(0, now - previous), MAX_FRAME_MS)

    const end  = scene.current.lastIndex
    const next = cursor.current
      + (delta / 1000) * PLAYS_PER_SECOND * scene.current.speed

    if (next >= end) {
      cursor.current = end
      render()
      setPlayIndex(end)
      stop()
      setPlaying(false)
      return
    }
    cursor.current = next
    render()
    setPlayIndex(Math.floor(next))
    frame.current = requestAnimationFrame(advance)
  },
  [render, setPlayIndex, setPlaying, stop],
)
```

CONSTANT	VALUE	REASONING
`PLAYS_PER_SECOND`	8	A typical 170-play game runs a little over twenty seconds — long enough to read the swings, short enough to watch twice. Playback is paced *by plays* rather than by a fixed total duration, so a long game genuinely feels long.
`SPEEDS`	0.5, 1, 2	Multipliers on the above.
`MAX_FRAME_MS`	100	The most time a single frame may advance the cursor. Six frames' worth: large enough to absorb an ordinary dropped frame, small enough that nothing lurches.

9.5 Five consequences, each of which was a bug first

1. Position is derived from elapsed time, never from a frame count

A dropped frame slows the replay by a frame's worth; it does not push it out of step with the clock. A frame-counting implementation drifts as soon as the machine is busy.

2. A single frame may advance the cursor by at most 100ms

`requestAnimationFrame` stops firing in a background tab, so the first frame after returning can carry *minutes* of wall time and would otherwise jump the replay straight to the final whistle. Capping the step means a hidden tab simply holds its place. There is a unit test for exactly this: **a 120-second frame advances 0.8 of a play, not the whole game.**

3. The canvas is measured in a `ResizeObserver`, never inside a frame

```
useEffect(() => {
  const canvas = canvasRef.current
  if (!canvas) return
  const observer = new ResizeObserver((entries) => {
    const box = entries[entries.length - 1]?.contentRect
    if (!box) return
    size.current = { width: Math.round(box.width), height: Math.round(box.height) }
    render()     // ResizeObserver reports the initial size when it begins
  })              // observing, so this doubles as the first paint
  observer.observe(canvas)
  return () => observer.disconnect()
}, [render])
```

Reading `getBoundingClientRect()` during a frame forces a synchronous layout whenever anything has dirtied the DOM — **which the scrubber does every frame by writing its own progress.** Moving the measurement took forced layouts during playback from 60 a second to none.

4. The scrubber thumb is written imperatively, from the same cursor

```
const stop = Math.round(fraction * 1000)     // a thousandth of the track
const play = Math.floor(cursor.current)
// A deferred write may leave the thumb a fraction of a pixel behind, but
// never a whole play behind: the readout beside it moves the instant the
// cursor crosses a boundary, and a deferral that straddles one made the
// slider report a different play from the text next to it.
if (stop !== sliderAt.current || play !== sliderPlay.current) {
  sliderAt.current   = stop
  sliderPlay.current = play
  slider.value = String(cursor.current)
  slider.style.setProperty('--progress', `${fraction * 100}%`)
}
```

Moving the thumb relayouts the control and refilling the track restyles it — together the most expensive thing outside the canvas. Writes the eye cannot see are skipped: a thousandth of the track is under a pixel wide at any width this control is ever given, and at 1× the cursor crosses one of those steps about 37 times a second rather than 60. **But the play-boundary term is not an optimisation — it is a correctness fix**, and there is a test named after the bug: `it('never reports a different play from the readout beside it', ...)`.

5. The drawing module is pure

A context, a size and a cursor go in; pixels come out. Nothing in it reads the clock, React state, or the DOM. The same code renders the running animation, a paused frame, and any position a scrubber is dragged to — and it can be tested against a recording context with no browser at all.

9.6 `replayCanvas.ts` — the pure drawing module

Coordinate mapping

```
export const PLOT_PADDING = { top: 12, right: 14, bottom: 24, left: 40 }
// Exported because the scrubber has to span exactly the plot, not the whole
// card: the thumb sits under the cursor line only if both use these insets.

export function xForIndex(index: number, lastIndex: number, plot: Plot): number {
  if (lastIndex <= 0) return plot.x + plot.width / 2
  return plot.x + (index / lastIndex) * plot.width
}

export function yForProb(prob: number, plot: Plot): number {
  const clamped = Math.min(1, Math.max(0, prob))   // bad data cannot draw off-canvas
  return plot.y + (1 - clamped) * plot.height
}

export function indexAtOffset(offsetX, lastIndex, size): number | null {
  const plot = plotArea(size)
  if (plot.width <= 0 || lastIndex <= 0) return null
  const fraction = (offsetX - plot.x) / plot.width
  if (fraction < -0.01 || fraction > 1.01) return null
  return Math.min(lastIndex, Math.max(0, Math.round(fraction * lastIndex)))
}
```

`indexAtOffset` is the exact inverse of `xForIndex`, and there is a test asserting that round-trip **for every play at three widths**. It takes the offset the event already carries, so hovering never measures the DOM.

> **WHY THE X AXIS IS PLAY ORDER, NOT ELAPSED TIME**
>
> Several plays legitimately share one timestamp — a kickoff and the snap after it are both logged at 15:00 — so an elapsed-seconds axis *collides* points that play order separates, and cannot guarantee one point per play. Play order gives every play its own position, and it is also the axis the replay itself advances along. This was a deliberate reversal: an early version plotted elapsed time, and the commit that changed it is titled “Stage 5A checkpoint: plot play order, not elapsed time”.

Device pixel ratio, folded in once

```
const MAX_DPR = 2   // Retina is worth paying for; a 3x phone screen is not.
                    // Backing-store pixels grow with the SQUARE of this number,
                    // and the third multiple buys nothing the eye can resolve
                    // on a two-pixel line.

export function applyCanvasSize(canvas, ctx, size: Size): void {
  const dpr = Math.min(window.devicePixelRatio || 1, MAX_DPR)
  const backingWidth  = Math.round(size.width  * dpr)
  const backingHeight = Math.round(size.height * dpr)

  // Assigning to canvas.width CLEARS the canvas, so it is guarded: an
  // unguarded assignment would wipe the drawing on every frame.
  if (canvas.width !== backingWidth || canvas.height !== backingHeight) {
    canvas.width  = backingWidth
    canvas.height = backingHeight
  }
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0)
}
```

All drawing code then works in CSS pixels and never thinks about the ratio again.

Interpolation at the tip of the line

```
export function probAt(points: readonly ChartPoint[], cursor: number): number {
  if (points.length === 0) return 0.5
  const lastIndex = points.length - 1
  const at = clampCursor(cursor, lastIndex)
  const whole = Math.floor(at)
  const here = probOfPlay(points, whole)
  if (whole >= lastIndex) return here
  return here + (probOfPlay(points, whole + 1) - here) * (at - whole)
}
```

The cursor advances continuously while plays are discrete, so the tip of the revealed line is interpolated between the play it has passed and the one it is heading for. Without this the line would grow in visible steps.

The NaN trap

```
function clampCursor(cursor: number, lastIndex: number): number {
  if (!Number.isFinite(cursor)) return 0
  return Math.min(lastIndex, Math.max(0, cursor))
}
```

Clamping alone does not remove a NaN — `Math.min(last, Math.max(0, NaN))` is still NaN — and it then travels into every coordinate, which the canvas skips *without complaint*: the curve simply disappears. Turning it into a position at the top is the only treatment that leaves something on screen. There is a test: “a non-finite cursor still draws”.

The frame, in order

```
export function drawFrame(ctx, options: FrameOptions): void {
  const plot = plotArea(options.size)
  const lastIndex = Math.max(0, options.points.length - 1)
  const cursor = clampCursor(options.cursor, lastIndex)

  ctx.clearRect(0, 0, size.width, size.height)   // safe at any cursor, any order
  drawBackground(ctx, plot, marks, lastIndex)    // grid, % labels, quarter marks
  if (points.length === 0) return
  drawCurve(ctx, points, cursor, lastIndex, plot, color)      // fill + stroke
  drawKeyPlays(ctx, points, keyIndices, cursor, lastIndex, plot, color)
  drawCursor(ctx, cursor, probAt(points, cursor), lastIndex, plot, color)
  if (hover !== null && hover >= 0 && hover <= lastIndex) drawHover(...)
}
```

Details worth knowing:

- **Key-play beads break early.** `keyIndices` is ascending, so the first index past the cursor ends the loop — no filtering pass per frame.
- **Beads are filled with the surface colour and ringed in the accent**, so they read as markers rather than as part of the curve. The cursor dot is ringed the same way so it stays legible where the line doubles back over itself.
- `crisp(v) = Math.round(v) + 0.5` for every 1px line, so it lands on one pixel instead of straddling two.
- **The 50% line is dashed and drawn at 3:1**, not at the faint gridline colour — it means something, so it has to be visible.

> **THE OPTIMISATION THAT WAS MEASURED AND REJECTED**
>
> The background — grid, labels, even-odds line, quarter dividers — does not change between frames, so it looks like an obvious candidate for an offscreen bitmap cache. Profiled during playback it is not worth one: **the labels cost 0.03 ms per frame and everything drawn on the canvas together costs 0.14 ms, against a 16.7 ms budget.** Blitting a cached bitmap of the same size would cost more than it saved. This is the single best answer to “tell me about a performance decision”: the interesting part is the one that was *not* made.

9.7 Transport semantics

ACTION	DEFINED BEHAVIOUR
`play()`	If the replay is finished, rewind to 0 first — that is what the button offers to do at that point. Then start.
`resume()`	Continue from the cursor, **without** the rewind-when-finished rule. Used when a drag ends.
`pause()`	Cancel the frame, publish `playing: false`.
`seek(position)`	Deliberately says *nothing* about whether the replay is running. Seeking while paused stays paused; seeking while playing carries straight on from the new position, because the loop reads the same cursor on its next frame.
`nudge(plays)`	Counts from `Math.floor(cursor)`, not from the fractional cursor, so a press always lands on a play boundary and repeated presses walk plays one at a time instead of accumulating a fraction.
`restart()`	Stop, cursor to 0, render, start.
`start()`	Always calls `stop()` first. **Never stacks two loops** — there is a test asserting no two loops ever run.

Scrubbing

```
function beginScrub() {
  // Playback is suspended for the length of a drag: letting the cursor run
  // on would move the target out from under the thumb. It picks up again
  // from wherever the drag ended, which is what a video scrubber does.
  resumeAfterScrub.current = playing
  if (playing) replay.pause()
}
function endScrub() {
  if (!resumeAfterScrub.current) return
  resumeAfterScrub.current = false
  replay.resume()     // resume, not play — a finished replay stays finished
}
```

Keyboard on the scrubber

The track is `step="any"` so the thumb can follow the cursor smoothly — which leaves the browser moving by a hundredth of the game per arrow press, a quantity that means nothing here. So the keys are intercepted: **one press is one play.**

```
const KEY_JUMP = { ArrowLeft: -1, ArrowDown: -1, ArrowRight: 1,
                   ArrowUp: 1, PageDown: -10, PageUp: 10 }
// plus Home → 0 and End → lastIndex
```

9.8 Reduced motion

```
cursor.current = reduceMotion ? lastIndex : 0
setPlayIndex(cursor.current)
if (reduceMotion) { setPlaying(false); render() } else { start() }
```

With reduced motion the replay renders **the finished curve immediately**, with the transport still there if you want to watch it play. It is not disabled — the preference is honoured, not used as an excuse to remove the feature.

9.9 Changing game mid-mount

```
useEffect(() => {
  stop()
  // Forget where the thumb was last written: a different game has a
  // different scale, so the skip-if-unchanged test must not match against
  // the previous game's position.
  sliderAt.current = -1
  sliderPlay.current = -1
  hover.current = null
  setHoverIndex(null)
  cursor.current = reduceMotion ? lastIndex : 0
  ...
  // Cancels the pending frame on unmount, AND before this effect re-runs for
  // a different game. Nothing else holds a frame handle.
  return stop
}, [game.gameId, lastIndex, reduceMotion, render, ...])
```

The `scene` ref mirrors `{ points, marks, keyIndices, color, lastIndex, speed }` so the loop can read the current game and colour **without being torn down and rebuilt mid-playback**. It is declared before every other effect so it is already current by the time they run.

9.10 The key-play rail

A game carries up to 41 markers. Forty-one tab stops between the scrubber and the Play button would be a wall, so the rail is **one tab stop with a roving tabindex** and arrow / Home / End navigation.

> **THE CLUSTERING PROBLEM, AND THE FIX**
>
> Key plays cluster — a touchdown, its extra point and the kickoff after it are consecutive — so in the densest game in six seasons the markers sit **a median of fifteen pixels apart on desktop and three on a phone**, well inside each other's hit area. Stacked 24px hit boxes hand the click to whichever marker happens to be later in the DOM: **aiming at the centre of each of forty-one markers landed on a different play nineteen times.**
>
> Rather than shrink the targets to the width of a tick, the rail resolves a pointer click to the *nearest* marker itself, in the **capture phase**, so it settles the click before any overlapping marker can.

```
function onRailClickCapture(event: React.MouseEvent<HTMLDivElement>) {
  // Keyboard activation arrives here too — Enter on a focused button
  // dispatches a click — but with a detail of 0, and that one is left alone
  // so the FOCUSED marker is the one that acts.
  if (event.detail === 0) return
  const rail = event.currentTarget.getBoundingClientRect()
  if (rail.width <= 0) return
  const wanted = ((event.clientX - rail.left) / rail.width) * lastIndex
  let nearest = 0, shortest = Infinity
  keyIndices.forEach((index, position) => {
    const distance = Math.abs(index - wanted)
    if (distance < shortest) { shortest = distance; nearest = position }
  })
  const index = keyIndices[nearest]
  if (index === undefined) return
  event.stopPropagation()
  setFocused(nearest)
  onSelect(index)
}
```

The corresponding E2E test clicks a marker **as a pixel rather than as an element** — because Playwright's actionability check refuses a click on an element a neighbour covers, while a real finger lands there anyway.

SECTION TEN

Feature walkthrough

10.1 League dashboard — `/`

Fetches: `teams-index.json` + `standings.json` + `meta.json` = 4.1 KiB gzipped, all three settled in one frame.

```
const state = useAsync<LeagueData>('league', async () => {
  const [teams, standings, meta] = await Promise.all([
    getTeamsIndex(),
    getStandings(),
    getMeta().catch(() => null),     // ← allowed to fail
  ])
  return { teams, standings, meta }
})
```

TWO BUGS ENCODED IN FIVE LINES

Not `Promise.all` over all three. `meta.json` supplies the season to open on and the coverage note; neither is load-bearing, because the seasons themselves come from the standings — the data actually being shown. An earlier version had it in `Promise.all` and a 0.2 KB file could take the whole dashboard down. **An E2E test that aborts that one request caught it at once.**

But fetching it separately bought a layout shift instead: the coverage note lands above the controls, so arriving a frame later pushed the season tabs, the controls and the whole grid down 50 px — measured at 0.026 CLS on the run where meta was slowest. Settling all three together with `.catch(() => null)` means the page paints once *and* survives a missing meta.

Interactions that never touch the network: sorting (three keys × two directions), filtering (conference, division), and changing season. All are `useMemo` over data already in hand. `standings.json` holds 192 rows for 2.6 KB precisely so that a season switch is a re-render.

URL state: `?sort=&dir=&conf=&div=&season=`, each omitted when it equals the default. Two rules in `viewFromParams`:

- Each sort key has its own natural direction (`projected: desc`, `record: desc`, `division: asc`). Defaulting every key to `desc` made a click on Division open at NFC West, which is not what anyone means.
- **A division that contradicts its conference loses.** `?conf=AFC&div=NFC%20West` would render an empty grid, so the division falls back to ALL — it is the narrower, more accidental choice.

Accessibility: the result sentence is an `<h2 aria-live="polite">`, not a paragraph — the cards below are `h3`, and without a level between them and the page title the outline skipped h1 to h3.

Animation: the grid uses `LazyMotion features={domAnimation} strict` with the lightweight `m` components, keyed on team id so re-sorting reorders existing DOM rather than remounting. `strict` makes reaching for a heavier `motion.*` component a runtime error rather than a silent bundle regression.

10.2 Team page — `/team/:id?season=`

Fetches: `teams-index.json` + `team/<season>/<ID>.json` = 5.2 KiB gzipped. The index is usually already cached from the dashboard; it supplies opponent names and logos for the games list.

Five sections in one scroll with an anchor nav: Games (first — they are the route into the replay, which is the point of the whole application), Team stats, Depth chart, Roster, Draft class.

THE SEASON CONTROL OUTLIVES THE SEASON IT SELECTS

Changing season refetches. A control rendered inside the loaded content therefore *unmounted while the next file arrived* — and pressing a tab with the keyboard dropped focus to `<body>`, leaving a visitor to tab in again from the top of the document.

The breadcrumb and the season control are now rendered from `meta.json`, which does not change between seasons, and only the body below them swaps for a skeleton. Both the control and the fetch behind it resolve the season through one `resolveSeason`, so the pressed tab cannot disagree with the file on screen.

Its accessible name is **"Season" rather than the team's**, because focus now stays on that button while the name would have resolved from "KC" to "Kansas City Chiefs" underneath it — renaming the group under whoever is standing on it.

The stat bars are drawn against a fixed printed scale, because a team file carries no league distribution to rank against. The domains are chosen to span a realistic season and are always printed, which is what stops a bar from implying a percentile it cannot know. The "Compare KC" button is the way to give those bars a second team to be read against.

10.3 Game replay — `/game/:id`

Fetches: one file, 6.6 KiB gzipped. Covered in full in Section 9. Presentation notes:

- **The scoreboard stacks on mobile.** A phone cannot fit two club names either side of a score: each side got 46 px and clipped. Mobile stacks the two teams; only `sm+` mirrors them across a centre column with `flex-row-reverse`.
- **The canvas carries `role="img"`** with a label naming both teams, the number of key plays and the final score — so a screen-reader user gets the summary the picture conveys.
- **The breadcrumb carries the season:** `/team/${game.home.id}?season=${game.season}`, so the way back from a 2020 replay is the 2020 team page.
- **The hover tooltip is `aria-hidden`** and decorative: the same play is one keyboard step away on the scrubber, which announces it properly. Only a mouse hovers — `if (event.pointerType !== 'mouse') return` — a touch gets tap-to-seek, with no tooltip to dismiss.

10.4 Compare — `/compare?a=&b=`

Nothing new is generated for this page. Both teams' numbers already live in their team files, and the head-to-head comes from `games-index.json`, which the ETL has always produced and both validators have always checked. This page is its first reader.

Fetches when a matchup is chosen: two team files + the 27 KiB game index = ~35 KiB. Keyed on the pair, so choosing a different opponent refetches exactly once and a half-chosen matchup issues no request at all.

- **Defensive metrics invert the leader** via `lowerIsBetter`, with "lower is better" printed next to the Defense heading.
- **Meetings are symmetric** in their arguments and never include a third team; the two series records mirror. All four properties are asserted by tests.
- **A team compared with itself drops the second slot**, because the answer would be five ties and no meetings.
- One `aria-live="polite"` region wraps the whole result, so choosing a team announces what is now on screen instead of nothing.
- `NO_TEAMS` is a single shared frozen empty array — a fresh `[]` per render would be a new identity every time, so the matchup memo would recompute on every render while the index is in flight.

10.5 Player search — `/players`

Fetches: `players-index.json` (73 KiB gzipped) + `teams-index.json`. All 2,274 entries are filtered *in the browser*. There is no server to ask, and filtering an array of that size is faster than the keystroke that triggered it — **a debounce**

would only add latency.

Normalisation

```
export function normalise(text: string): string {
  return text
    .normalize('NFD')
    .replace(/[̀-ͯ]/g, '')   // strip combining accents
    .toLowerCase()
    .replace(/[^a-z0-9]/g, '')          // strip punctuation
}
```

NFL rosters are full of names a keyboard does not reach directly — Ja'Marr, Amon-Ra, Nuñez — and someone searching for them types "jamarr", "amonra", "nunez". **This is the difference between the search working and the search being a party trick that only finds Smith.**

Four-tier ranking

TIER	RULE	EXAMPLE
0	the surname starts with it	"all" finds Josh **All**en
1	any other name part starts with it	"jos" finds **Jos**h Allen too
2	the whole name, spaces removed, starts with it	"joshal"
3	it appears somewhere	"add" still finds W**add**le, but last

Substring matching alone put "Jaylen Waddle" above "Josh Allen" for the query "all", which is wrong: **a surname beginning with what you typed is almost always what you meant.**

Within a tier, whoever has played more. Two players share a surname often enough to matter — "jefferson" finds both Justin and Jermar — and with no popularity signal in the dataset, recorded career games is the honest stand-in for which one was meant. Alphabetical order, which is what the index arrives in, put the wrong Jefferson first.

> **THE SEARCH BOX KEEPS ITS OWN STATE; THE URL MIRRORS IT**
>
> Driving a text input from the query string looks tidy and **drops characters**. `setSearchParams` is a navigation, so on a fast typist every keystroke handler reads a filter from a render that has not happened yet and writes over the one before it. **Typing "garrett" left "t" in the box.**
>
> The two `<select>` facets are still URL-driven — one change per interaction cannot race itself. This asymmetry is deliberate and commented.

60 results are rendered at a time, because every one carries a headshot and rendering all of them would ask the browser for two thousand images to satisfy a search someone is still typing. The count above the list always reports the true total — and it is placed *above* the list, not below, because someone reading top to bottom (a screen reader especially) needs that said before the list, not after they have walked it.

10.6 Player profile — `/player/:id?season=`

- **Which numbers appear is decided by which numbers exist** (`groupsIn(line)`), never by the position in the file. Five groups: passing, rushing, receiving, defense, kicking.
- **`primaryGroup(weeks)` picks the weekly chart's metric** by whichever group has the most weeks of production — so a running back who caught two passes is still charted on carries, and a punter who once threw a pass is not suddenly a quarterback.
- **One season at a time, except where the long view is the point.** A profile carries every season's weekly rows — a hundred bars for a six-season career, which is not a chart — so the season control governs the stat groups and the

weekly trend together, and **the career table below them deliberately stays whole.** Six seasons of weekly logs cost about 0.4 KB gzipped, the cheapest thing in the repository by some margin.

- **A career row links to that team in that year** — `/team/:id?season=`, carrying the year from the row.

10.7 The shell

- `min-h-dvh flex flex-col` with a skip link, header nav, `<main id="main">`, and a footer carrying the nflverse attribution the CC BY 4.0 licence requires.
- **Suspense is inside the shell**, so header, nav and the attribution stay mounted while a lazily-loaded page chunk downloads.
- **The page transition is nine lines of CSS**, keyed on pathname so React replaces the subtree and the animation restarts. Only opacity and a few pixels of travel move, so the incoming page is interactive from its first frame. It used to be Framer Motion — see Section 13.4.
- **`DataFreshness` is lazy with `fallback={null}`**: the footer line simply appears when it can. Reserving space for it would trade a real layout shift for an empty gap, and a wrong freshness claim is worse than none.

SECTION ELEVEN

The six-season refactor

The dataset always held six seasons of replays. Everything *around* them described one. This is the most interesting refactor in the project and it has a counter-intuitive result.

11.1 The problem

The dashboard, the rosters, the stat lines and the records were all the newest season, so **the only way to look at the 2020 Chiefs was to read their 2020 scores underneath their 2025 squad.**

11.2 The change, and the surprise

The team layer became **one file per team per season** — 192 of them where there were 32. A visitor still fetches exactly one, and **it got smaller**: 5.9 KB gzipped to 3.8 KB, because it no longer has to carry six seasons of games to make them reachable from a single page.

PAGE	FETCHES	BEFORE	AFTER
League dashboard	teams-index (+ standings)	1.4 KB	**4.0 KB**, all six seasons
Changing season on it	nothing	—	**0 bytes** — a re-render
Team page	teams-index + the team file	7.3 KB	**5.2 KB**
Player page	teams-index + the profile	1.9 KB	**2.3 KB**

Gzipped, excluding `meta.json`, which the footer fetches on every page either way. The player figure is an average; Mahomes, who has played every season in the dataset, is the outlier at 3.7 KB.

The repository grew from 90 MB to 99 MB, which is the part that actually costs something — though 79 MB of it was replays before any of this. *That is the trade: repository size for per-visit size and correctness.*

11.3 The season has to survive leaving the page

League → Team → Game → Replay is the spine of this site, and a season that only existed on the team page broke it on the way back: **a 2020 replay's breadcrumb dropped a visitor on this year's Chiefs.**

Every link into a team page now carries the season it means:

- the replay's breadcrumb, from the game's own season
- a career row on a player page, from the year in that row
- the dashboard card, from the season on screen

11.4 One rule, one function

```
export function resolveSeason(meta: Meta, asked: string | null): number {
  const wanted = Number(asked)
  return meta.teamSeasons.includes(wanted) ? wanted : meta.displaySeason
}
```

One function rather than two, **because the team page resolves this twice** — once to render the control and once inside the fetch that answers it — and two copies of the rule are two chances for the pressed tab to disagree with the file on screen. Anything with no team layer falls back: a hand-edited URL, a bookmark from before a refresh, or a season still too young to describe.

SECTION TWELVE

Accessibility

Lighthouse scores accessibility 100 on all five routes, on both form factors, in all thirty runs — but Lighthouse's automated checks are a floor, not the audit. The audit was run with the keyboard only, and the numbers came from the rendered page rather than from reading the CSS.

12.1 Contrast is measured against the composited background

> **WHY A REGEX WAS NOT GOOD ENOUGH**
>
> Tailwind v4 serialises colours as `oklch()`. Parsing the stylesheet with a regular expression produced **60+ false failures** before the approach changed. Each colour is now painted into a canvas and read back, so the value under test is the value that reaches the screen.
>
> Two real failures were found and fixed: `text-neutral-600` at **2.46:1** and `text-neutral-500` at **4.05:1**. Both are short of the 4.5:1 normal-size text owes. The replacement is a single theme token, `--color-muted: #858585`, the lightest grey that clears 4.5:1 on every surface in the app — the page, a card, and a bar track. **All 88 distinct text styles across four pages now clear 4.5:1.** The faintest tier separates itself by size and weight rather than by fading further.

12.2 Graphics that carry meaning clear 3:1

WCAG 1.4.11. NFL primary colours span nearly the whole luminance range — New Orleans' gold (`#D3BC8D`) manages 1.85:1 against white, Pittsburgh's black gives 21:1 — so any colour placed on a surface has to be chosen by measurement.

`legibleOn` — for the replay curve

```
export function legibleOn(surface, color, minimum = GRAPHIC_CONTRAST): string {
  const rgb = parseHex(color)
  if (!rgb) return '#a3a3a3'
  for (let mix = 0; mix <= 1; mix += 0.02) {
    const lifted = toHex([ rgb[0] + (255 - rgb[0]) * mix,
                           rgb[1] + (255 - rgb[1]) * mix,
                           rgb[2] + (255 - rgb[2]) * mix ])
    if (contrastRatio(lifted, surface) >= minimum) return lifted
  }
  return '#ffffff'
}
```

Nine of the 32 primary colours miss 3:1 against the card, the Jets' green managing 1.65:1. Rather than drop those teams to a neutral — which would make a third of the league draw an identical grey line — the hue is mixed toward white until it clears the bar. Every team gets there while still looking like itself. Black is the exception it cannot help: Las Vegas and Pittsburgh have no hue to keep, so they arrive at grey either way.

`teamAccent` — for bar fills

```
export function teamAccent(track, primary, secondary): string {
  if (contrastRatio(primary,   track) >= GRAPHIC_CONTRAST) return primary
  if (contrastRatio(secondary, track) >= GRAPHIC_CONTRAST) return secondary
  return legibleOn(track, primary)
}
```

Twelve of the 32 primaries miss 3:1 against `BAR_TRACK` (`#262626`), Buffalo's royal blue managing 1.34:1. The secondary colour is tried *before* lightening, because a team's own second colour keeps more of its identity than a washed-out first one — Pittsburgh gets its gold rather than a pale black.

THE SURFACE CONSTANT THAT MATTERS

`CARD_SURFACE = '#0f0f0f'` is `bg-neutral-900/40` composited over `bg-neutral-950` — (0.4 × 23) + (0.6 × 10) per channel. Contrast for anything drawn on the canvas has to be measured against *this*, not against the page behind it, or every colour comes out a little brighter on paper than it looks on screen.

Both rules are enforced by tests that read **the real team index**, because nine and twelve clubs respectively failed before those tests existed.

12.3 Keyboard operation

PROPERTY	RESULT
Tab stops, League	44 — all reachable, all with a visible focus ring
Tab stops, Team	30
Replay at 320px	Fully operable
Focus traps	None. Focus escapes every page.
Key-play rail	One tab stop, roving tabindex, arrow / Home / End
Scrubber	Arrows = 1 play, PageUp/Down = 10, Home/End = ends
Skip link	`sr-only focus:not-sr-only`, first in the DOM

RESTART IS NEVER DISABLED

Disabling it on reaching play 0 — which pressing it does — **moved focus to `BODY`**, so a keyboard user lost their place the moment the button worked. Restarting an already-restarted replay is harmless. There is a test named after it: `it('keeps Restart enabled, so pressing it never drops focus to the body', ...)`.

12.4 Native semantics are not re-labelled

Buttons are buttons. The scrubber is an `<input type="range">`. ARIA appears only where there is no native equivalent.

The scrubber's raw value is a play index, which on its own says nothing about where you are in the game — so `aria-valuetext` carries the period, the clock and the score:

```
"Play 42 of 218, Q3 6:12, NO 17, ATL 24, ATL 71 percent"
```

rather than "42".

`SeasonTabs` uses `role="group"` + `aria-pressed` rather than a tablist, **because these do not reveal panels** — they filter one list that is always there. Announcing a tab that swaps panels would describe something the page does not do.

12.5 `prefers-reduced-motion` is honoured everywhere

- The replay renders the finished curve immediately, transport still available.
- The count-ups return their final value at once (`useCountUp` returns `target` directly).
- The grid stagger goes to 0 and the section reveal to `initial={false}`.
- The skeleton pulse stops.

SECTION THIRTEEN

Performance

Everything below is measured, not estimated. Lighthouse 13.4.1 against **the deployed site**, headless Chrome, desktop preset and the default mobile preset (4× CPU throttling, simulated slow 4G). Machine benchmark index 3057.

> **THREE RUNS PER ROW, AND WHY THAT MATTERS**
>
> Scores are the **median** and the spread is printed, because a single mobile run is not a measurement: the same route came back **93 and 99 twenty seconds apart.** An earlier version of this table was single runs and claimed repeats moved "by at most one point", which thirty runs did not bear out. *Correcting your own published claim with more data is a good thing to be able to say in an interview.*

13.1 Lighthouse

PAGE	FORM	PERF	A11Y	BP	SEO	FCP	LCP	TBT	CLS	WEIGHT
League	desktop	100	100	100	100	0.4 s	0.5 s	0 ms	0	274 KiB
Team	desktop	100	100	100	100	0.3 s	0.4 s	0 ms	0	178 KiB
Game	desktop	100	100	100	100	0.4 s	0.4 s	0 ms	0	132 KiB
Compare	desktop	100	100	100	100	0.4 s	0.4 s	0 ms	0	155 KiB
Players	desktop	100	100	100	100	0.4 s	0.4 s	0 ms	0	307 KiB
League	mobile	100 (96–100)	100	100	100	1.3 s	1.8 s	1 ms	0	225 KiB
Team	mobile	96 (96–97)	100	100	100	1.5 s	2.1 s	40 ms	0	178 KiB
Game	mobile	99 (97–100)	100	100	100	1.4 s	1.6 s	18 ms	0	132 KiB
Compare	mobile	97 (93–99)	100	100	100	1.4 s	1.8 s	0 ms	0	155 KiB
Players	mobile	95 (92–96)	100	100	100	1.4 s	2.4 s	10 ms	0	287 KiB

Desktop is 100 on every route, in all fifteen runs. So is accessibility, best practices and SEO, on every run of both forms.

`/players` is the heaviest page and the slowest on mobile — 287 KiB, LCP 2.4 s — because it carries the 73 KiB search index and sixty headshots. That is the price of a list you scan by face rather than by name, and dropping the thumbnails would be a one-line change if it stopped being worth it.

The same build measured on `vite preview` over localhost scores *lower* on mobile: the CDN's brotli and HTTP/2 are doing real work, so a local preview is the pessimistic reading rather than the flattering one.

13.2 Cumulative layout shift is 0 everywhere — after two self-inflicted fixes

The footer

The app shell is `min-h-dvh flex flex-col`, so while a skeleton is up the page is exactly one viewport tall and the footer sits visible at the bottom of it. When the real content lands — 2,009 px for the dashboard — the footer is pushed off, and **that displacement was most of the measured shift.** Skeletons now reserve 120vh, which every route in this app overruns anyway.

The coverage note

The dashboard's coverage note arrived a frame after the standings and pushed 50 px of content down. The three files it needs now settle together, with the freshness file allowed to fail. **Delaying `meta.json` by two seconds against the**

production build now produces 0.0000.

> **A REVERSED ARGUMENT — WORTH TELLING AS A STORY**
>
> An earlier version of the README argued the other way: that the team page's CLS of 0.069 was just the footer moving below the fold, that the figure was inside Google's "good" threshold, and that "padding the skeleton until the footer starts off-screen would improve the metric and not the experience".
>
> That was wrong on two counts. **Reserving 120vh is not padding to move a number**: every route in this app loads to more than one screen, so the space is genuinely going to be used, which is the whole job of a skeleton. And the argument was doing double duty as an excuse — it sat next to a second, unambiguous shift where the coverage note pushed the tabs, the controls and the grid down 50 px, which no threshold excuses. CLS is now 0 on every route and form.

13.3 Frame rate

240 consecutive frames during playback of the longest game in the dataset (2022 week 15, IND at MIN, 218 plays, overtime), in the production build:

```
median frame  16.70 ms      p95  16.70 ms      worst  16.80 ms
frames over 33 ms (a dropped frame): 0 / 239
long tasks (>50 ms) during 3 s of playback: 0
```

That is the 60 Hz budget, held. Profiled separately: **all canvas drawing costs 0.14 ms per frame and the axis labels 0.03 ms** of a 16.7 ms budget.

13.4 Bundle

```
index               267.36 kB   84.78 kB gzip   React, Router, the shell — every page
features-animation   73.87 kB   25.80 kB gzip   Framer Motion — League and Team only
CSS                  30.73 kB    6.51 kB gzip
GamePage             17.85 kB    6.29 kB gzip   the replay engine — only on a game page
TeamPage             16.48 kB    4.71 kB gzip
ComparePage          10.61 kB    3.51 kB gzip
LeaguePage            7.66 kB    2.70 kB gzip
useAsync              7.38 kB    2.98 kB gzip   data layer + runtime contract
football              2.53 kB    1.25 kB gzip
league                2.41 kB    0.95 kB gzip
stats, motion hook    0.85 kB    0.53 kB gzip
```

Lazy loading verified from the network log, not from the config:

ROUTE	JAVASCRIPT	DATA	MOTION RUNTIME
`/`	113.5 KiB	4.1 KiB	yes
`/team/:id`	116.9 KiB	5.5 KiB	yes
`/compare`	90.5 KiB	1.4 KiB	**no**
`/game/:id`	92.3 KiB	6.6 KiB	**no**

`GamePage` appears on no other route's list. **The replay engine is downloaded when, and only when, someone opens a game.**

FRAMER MOTION IN THE SHARED LAYOUT COST 26 KB ON EVERY FIRST VISIT

Importing it into the layout for *one page-transition fade* put it in front of every first visit, **including game pages that never used it** — the entry chunk went from 84 kB to 110 kB. That transition is now nine lines of CSS, and the motion runtime is confined to the League and Team chunks, loaded with `LazyMotion features={domAnimation} strict` and the lightweight `m` components.

13.5 Images: two orders of magnitude, twice

Team logos — 1,789 KiB to 262 KiB

nflverse stores ESPN's 500×500 master, 42 to 79 KB per club. The league dashboard drew 32 of them at 40 CSS pixels: measured at **1,789 KiB of image for one page**, larger than the whole JavaScript bundle by an order of magnitude. ESPN's own image combiner resizes on their CDN:

```
export function logoAt(url: string, cssPixels: number): string {
  if (!url.startsWith(`${ESPN_ORIGIN}/`)) return url      // pass anything else through
  let path: string
  try { path = new URL(url).pathname } catch { return url }
  const size = Math.round(cssPixels * MAX_DPR)            // MAX_DPR = 2
  return `${ESPN_ORIGIN}/combiner/i?img=${path}&w=${size}&h=${size}`
}
```

The same page now weighs **262 KiB**. Nothing is proxied or re-hosted, and a URL from any other origin passes through untouched. **This belongs in the frontend rather than the ETL** because the size wanted is a property of where the logo is being rendered, not of the team — the stored URL stays the canonical, full-resolution one.

Player headshots — the surprise, twice

3.8 MB FOR A 36-PIXEL THUMBNAIL

nflverse points at the NFL's Cloudinary CDN, where the stored images are **3–6 MB PNGs**. One headshot is larger than every script, stylesheet and data file this site serves put together. They sit behind an `f_auto,q_auto` transformation, and appending a width resizes on their CDN: measured across four players, **3–5 MB becomes 6 KB at 160px and 20 KB at 320px.**

The second surprise was that the first fix matched only one of two delivery types. 55 of the 2,269 stored URLs come from `/image/private/` rather than `/image/upload/`, and those went out at their stored size: **A.J. Brown's 36-pixel thumbnail was a 3.8 MB PNG**, and the default `/players` listing carried **775 KB** of images. The same transform works on both paths — `w_72` makes that image 2.5 KB — so the repair was one regular expression.

Why it shipped is the more useful part. `headshotAt` had no unit test at all, and the E2E check ran against `?q=allen`, a search narrow enough to contain none of the 55. Three things changed: the E2E test now loads the *unfiltered* listing and fails over 30 KB for a thumbnail or 400 KB for the page; `validate.py` rejects any stored headshot the frontend cannot resize; and `headshotAt` has tests of its own.

Measured on a Pixel 5 viewport after the fix: the default listing fetches 51 headshots for 80 KB, largest 2.3 KB, and A.J. Brown's profile went from 3.8 MB to 6.1 KB.

```
const HEADSHOT =
  /^(https:\/\/static\.www\.nfl\.com\/image\/(?:upload|private)\/)f_auto,q_auto\/(.+)$/
//                                          ^^^^^^^^^^^^^^^^^^ the one-word fix

export function headshotAt(url: string, cssPixels: number): string {
  const match = HEADSHOT.exec(url)
  const prefix = match?.[1], asset = match?.[2]
  if (prefix === undefined || asset === undefined) return url
  const width = Math.round(cssPixels * MAX_DPR)
  return `${prefix}f_auto,q_auto,w_${width}/${asset}`
}
```

13.6 What is left on the table, and why

ITEM	COST	WHY IT STAYS
`unused-javascript`	36 KiB	React and Router code not executed during first paint. Real, but not separable without shipping a different framework.
Third-party cache headers	~48 KiB	The logos are served by ESPN's CDN with their cache policy, not ours.
The comparison uses the displayed season only	—	Both sides come from `meta.displaySeason`, so "Compare KC" from a 2020 team page opens a 2025 comparison. Nothing on the button claims otherwise, and `getTeamSeason` already takes a season — but comparing two teams in a chosen year needs its own control on that page to avoid being invisible state in a URL. **That is a feature rather than a fix.**

SECTION FOURTEEN

Testing strategy

```
389 unit and component tests   20 files    Vitest + React Testing Library
 40 end-to-end specs           x2 devices  Playwright (desktop Chrome, Pixel 5)
 16 fault injections                       validate.py, verified by corruption
 16 fault injections                       refresh_rules.py, stdlib only
```

The tests are aimed at behaviour that could plausibly break, not at a coverage number. That sentence is worth saying out loud in an interview, because the follow-up question is always "what's your coverage?" and the honest answer is that coverage is an input, not a goal.

14.1 What is actually asserted

AREA	ASSERTIONS
League sorting / filtering	Every sort key and direction, the name tiebreak, immutability of the input, contradictory URL params
URL state	Round-trips, unknown values falling back, a division that contradicts its conference losing
Data layer	404 → not-found; HTML-with-a-200 → not-found; truncated JSON → malformed; contract violation → malformed
Request cache	Concurrent callers share one fetch; failures are evicted and retryable
Contract	The *real generated files* parse; each failure mode names its path; extra fields are tolerated
Replay control state	Seek-while-paused stays paused; seek-while-playing continues; Play rewinds a finished replay but resume does not; **no two loops ever run**; the frame handle is cancelled on unmount
Frame pacing	A 120-second frame (a backgrounded tab) advances **0.8 of a play**, not the whole game
Canvas drawing	`indexAtOffset` inverts `xForIndex` for every play at three widths; beads appear only once passed; a non-finite cursor still draws
Colour	All 32 *real* team colours clear 3:1 on the card and on the bar track, and AA as badge text
Comparison	The leader is inverted for defensive metrics; an exact tie names no one; meetings are symmetric in their arguments and never include a third team; the two series records mirror
Components	Loading, error, retry, empty and not-found states; keyboard operation of the whole transport
Player search	Ranking tiers and the namesake tiebreak *against the real index*; accented names typed plainly; the URL restoring box, filters and results; both headshot delivery types resized
Six seasons	The standings join drops a team with no row for that year rather than showing it 0-0; a season only reaches the URL when it is not the default; the team page fetches the season it was asked for and falls back for one with no layer; the dashboard changes season without a request
Season continuity	Focus stays on the tab that was pressed across the refetch, and the control stays usable while it is loading; a replay's breadcrumb and a career row both carry the season they belong to; `resolveSeason` is one rule so the pressed tab cannot disagree with the file

14.2 Two tests that exist because the bug happened

```
it('keeps Restart enabled, so pressing it never drops focus to the body', ...)
it('never reports a different play from the readout beside it', ...)
```

Naming a test after the defect rather than after the function is deliberate: it tells the next reader *why the line below it cannot be simplified.*

14.3 Tests read the real generated data

Several suites load the committed JSON off disk rather than a fixture — the contract tests, the colour tests, and the player-search ranking tests. This is why `tsconfig.test.json` grants Node types that `tsconfig.app.json` deliberately withholds:

```
// Tests read fixture files off disk, so they get the Node types the
// application deliberately does not: nothing in src/ outside these files
// is allowed to reach for `node:fs`.
"types": ["vite/client", "node"]
```

The value of this is that a data refresh that breaks the app fails the *unit* suite, not just the validator — which is exactly what the weekly refresh workflow relies on.

14.4 End-to-end

Six specs, 40 tests, run against the production build served by `vite preview`, on desktop Chrome and a Pixel 5.

SPEC	COVERS
`replay.spec.ts`	The whole spine: league → team → game → replay; playback actually runs; scrubbing changes the context; seek does not restart playback; dragging leaves the replay where it was dropped; selecting a key play; full keyboard operation; a game not in the dataset says so
`seasons.spec.ts`	Six seasons offered; changing season fires **no request**; the season reaches the team page and the whole page follows; it survives a reload; an older roster links only profiled players; the season survives a trip into a replay and back; focus stays on the control
`compare.spec.ts`	Shareable matchup links; every metric names a leader or neither but never both; head-to-head meetings open replays; the team page pre-fills the comparison; a nonsense matchup degrades to empty pickers; keyboard operation
`players.spec.ts`	Finding a player from the header; punctuated names typed plainly; URL state; team filtering; a search matching nobody offers a way back; **every thumbnail on the default listing is sized**; the private-delivery headshot is resized too; keyboard operation keeps every key pressed
`player.spec.ts`	Rosters link only what has a page; a profile says more than the roster row; headshot byte budget; a week in the trend opens that replay; a player id with no profile says so
`data-freshness.spec.ts`	Every page says when the data was refreshed, with a machine-readable `<time>`; the league page says which season the figures describe; a team page names the season its stat lines come from; **the freshness note never blocks the page it sits under** (the request is aborted and the page must still be complete)

> **THE MOST INTERESTING E2E TEST IN THE SUITE**
>
> One test clicks a key-play marker **as a pixel rather than as an element**. Markers cluster, so on a phone a neighbour's 24px hit box covers the target and Playwright's actionability check refuses the click — while a real finger lands there anyway. The rail resolves clicks to the nearest marker for exactly that reason, and that is what the test is for. It is a case where the test framework's safety check was *modelling the wrong user*.

14.5 Fault injection on the Python side

`etl/test_refresh.py` is stdlib-only and imports `refresh_rules`, not `build_data`, so it runs with a bare Python and no install — importing the pipeline would have dragged `nflreadpy` in with it. It proves the two decisions the refresh makes unattended are safe:

```
check("the newest complete season wins",
      choose_display_season([season(2024, 285, 285),
                             season(2025, 285, 285)]) == 2025)

check("a barely-started season does not take over",
      choose_display_season([season(2025, 285, 285),
                             season(2026, 272, 2)]) == 2025,
      "two games is not a season")

raises("a rebuild that loses a game is refused",
       lambda: merge_index(existing_with_game, rebuild_without_it, {2026}))
```

SECTION FIFTEEN

CI/CD and the weekly data refresh

15.1 CI — three parallel jobs on every push and PR

JOB	STEPS
`verify`	`npm ci` → typecheck → lint → format check → 389 tests → production build → **bundle size report into the step summary**
`data`	`etl/validate.py` against the committed JSON, then `etl/test_refresh.py`. **Stdlib only — no ETL run, no install, no nflverse download.**
`e2e`	Playwright against the built app; HTML report uploaded as an artifact with 7-day retention, `if: !cancelled()`

A `concurrency` group cancels an in-flight run when a new push lands on the same ref. The bundle size is reported on *every* run rather than discovered at release, **because the bundle is the product here — there is no server to hide behind.**

15.2 The weekly refresh

```
Tuesday 09:00 UTC (or a manual run)
        │      ← after Monday night's game, and after nflverse publishes
        │        its play-by-play, which lags the final whistle by hours
        ▼
  python etl/build_data.py --refresh        rebuild the newest season only
        │
        ▼
  python etl/validate.py                    fail closed on a contract breach
  python etl/test_refresh.py                fault-inject the guardrails
  npm test && npm run build                 the app can still read it
        │
        ▼
  pull request with the diff                ← the one human step
        │
        ▼
  merge → Vercel deploys
```

Only the newest season is rebuilt

A full build re-derives six seasons and rewrites all 4,166 files. A refresh rebuilds the newest season and **merges** it into the committed index, so finished seasons come out *byte-identical*. It rebuilds the team layer only for seasons whose play-by-play it downloaded: a completed season's roster, draft class and stat lines never change again, so rewriting them weekly would be churn with no content.

```
Refresh: rebuilding 2026 only, merging into 1693 indexed games
  2025: 285/285 played, complete  <- team layer
  2026: 2/272 played, IN PROGRESS
Done. 32 teams, 1694 games, 1.0 MB total
```

`merge_index` will not let a refresh delete history

```
def merge_index(existing, rebuilt, seasons):
    kept = [g for g in existing if g["season"] not in seasons]
    was  = {g["gameId"] for g in existing if g["season"] in seasons}
    now  = {g["gameId"] for g in rebuilt}
    if lost := was - now:
        raise SystemExit(
            f"{len(lost)} game(s) present in the old index are missing from "
            f"the rebuild ({', '.join(sorted(lost)[:5])}). Refusing to drop them.")
    merged = kept + rebuilt
    merged.sort(key=lambda g: (g["season"], g["week"], g["date"], g["gameId"]))
    return merged
```

A game in the old index that the rebuild no longer produces **aborts the run rather than disappearing** — nflverse withdrawing a game, or a half-finished download, would otherwise silently remove replays that team pages still link to. *Rebuilding the index from just the rebuilt season is the single most destructive thing this script could do, so it is not possible to express.*

Which season the figures describe is a rule, not a judgement call

`PROMOTE_MIN_PLAYED = 64`

The team layer — rosters, stat lines, projections — describes exactly one season. Pointing it at a season two games old does not degrade gracefully, **it collapses.** On 11 September 2026, with two games played, it would have produced:

```
teams with a 2026 record: 4 of 32     the other 28 dropped from the dashboard
LA 0–1  projectedWins 0.60            next to a lastSeason record of 11–6
```

64 games is roughly four weeks — enough for a stat line to mean something and for every team to have played. Until a new season clears it, the team layer stays on the last complete one and the site says a newer season is under way. `--display-season` overrides by hand, **and a separate guard refuses to publish a dashboard missing teams whatever the flag says:**

```
if len(teams_index) < len(team_meta):
    raise SystemExit(
        f"Only {len(teams_index)} of {len(team_meta)} teams have a {current} "
        f"record. Refusing to publish a dashboard missing teams; the promotion "
        "rule should have kept the team layer on an earlier season.")
```

Nothing is presented as something it is not

```
{
  "generatedAt": "2026-09-11T20:13:01Z",
  "source": "nflverse",
  "displaySeason": 2025,
  "latestSeason": 2026,
  "teamSeasons": [2020, 2021, 2022, 2023, 2024, 2025],
  "seasons": [ ..., { "season": 2026, "scheduled": 272, "played": 2,
                      "complete": false } ]
}
```

`generatedAt` is truncated to **whole seconds** — a timestamp with microseconds in it changes on every run and would make an unchanged refresh look like a change.

`teamSeasons` is the same promotion rule applied to *which* seasons the site offers, not just which one it opens on. 2026 has a schedule and two replays but no squad worth describing, so it gets no team layer, no standings rows and no place on the season control — and `validate.py` fails if a season directory exists that this list does not name, or if the list names one that is missing.

Four things read `meta.json`

READER	BEHAVIOUR
Every footer	“Refreshed 4 minutes ago — 2026-09-11. Seasons 2020–2026.” Deliberately coarse; the exact instant is on the `<time>` element. Lazily loaded with no fallback, so a missing `meta.json` removes the line and nothing else.
The league page	Above the controls, not in a footnote, **because it changes how every number below it should be read.** It appears only when there is a distinction to explain.
Team stat panels	“From 1,048 regular-season scrimmage plays in 2025.” These are season-shaped figures and a reader who assumes they are this week’s is reading them wrong.
The team page	For the seasons its control offers and the one to open on. **The league dashboard deliberately does not** — its seasons come from `standings.json`, the data it is actually showing, so aborting `meta.json` leaves the whole dashboard working. An E2E test aborts that request and asserts exactly that; it caught this regression the day the season control was added.

Why a pull request and not a push

The data is the product; a bad refresh is a bad deploy. Everything up to the PR is automatic and fails closed — an invalid file, a dropped game, or an app that can no longer parse its own data all stop the run. **The diff is the review:** a handful of changed files is a normal week, and a larger one is a question worth answering first.

A run that changes nothing opens no PR, because a weekly “no change” PR trains people to merge without looking — which is the exact failure mode the approval step exists to avoid.

The PR body names what to look at: the file count in the run summary, and whether `displaySeason` changed. A change there means the team layer has moved to a new season, and every figure on the league and team pages now describes that season instead.

15.3 What the refresh does not do yet

Unplayed fixtures are not published. The ETL emits games that have both a result and play-by-play, so a schedule of upcoming games does not exist in the JSON — `GameSummary.homeScore` is a required number, and an unplayed game has no score to put there.

That work becomes necessary exactly when the promotion rule fires and the displayed season is one still being played. It is a *contract change* (optional scores, so absent keeps meaning absent) rather than a new file. Until then there are no future fixtures in the displayed season to show.

SECTION SIXTEEN

Deployment and caching

`vercel.json` cannot carry comments — Vercel validates it with `additionalProperties: false`, so even a `"comment"` key fails the build. The reasoning therefore lives in the README, and here.

```
{
  "rewrites": [
    { "source": "/((?!data/).*)", "destination": "/index.html" }
  ],
  "headers": [
    { "source": "/data/game/(.*)",
      "headers": [{ "key": "Cache-Control",
        "value": "public, max-age=86400, stale-while-revalidate=604800" }] },
    { "source": "/data/((?!game/).*)",
      "headers": [{ "key": "Cache-Control",
        "value": "public, max-age=3600, stale-while-revalidate=86400" }] }
  ]
}
```

16.1 The SPA fallback deliberately excludes `/data`

Without the exclusion a missing JSON file returns `index.html` with a **200**, and the fetch layer reports a parse error instead of "not found".

16.2 `stale-while-revalidate`, not `immutable`

Game files are not content-hashed and *do* change when the ETL is re-run, so `immutable` would strand visitors on stale data. Game files get a day of freshness and a week of stale-serving; the small index files get an hour.

> THE BUG PRODUCTION FOUND AND LOCAL DEVELOPMENT COULD NOT
>
> **Vercel's `headers` are cumulative, not first-match-wins like `rewrites`.** Every matching rule is applied, and the last one wins for a repeated key.
>
> Written as a plain `/data/(.*)`, the hour-long index rule **silently overwrote** the day-long game rule, and the header the deployed site actually returned for a 46 KB game file was the wrong one. This was invisible locally — `vite preview` reads no `vercel.json` — and was only caught by `curl`-ing the deployment.
>
> The fix is the negative lookahead in the second rule: `/data/((?!game/).*)`. **The second rule has to exclude what the first one matched.**

Commit: *"Fix the game-file cache header, which production was not applying"*. The lesson generalises: configuration whose semantics differ between local and deployed environments has to be verified against the deployment, not against the config file.

SECTION SEVENTEEN

War stories

Fourteen real bugs. Interviewers ask “tell me about a hard bug” and the good answers all have the same shape: the symptom, the wrong first theory, the actual cause, and what changed so it cannot recur. Pick three or four of these and know them cold.

#	SYMPTOM	CAUSE	FIX AND SAFEGUARD
1	Replay scores jumped *backwards* mid-game	`play_id` is not chronological: penalty and replay-review rows carry a later id than the snap they belong to	Sort by quarter, then game clock descending, then id. `validate.py` check 6 asserts chronological order on every file.
2	Scoreboard reset to 0–0 mid-replay	Timeout rows are logged as `no_play` and carry a stale score — ~2,200 per season	Dropped at the filter, plus a running max over both scores as a second line of defence. Validator asserts monotonic scores ending on the real final.
3	KC’s 2023 offense read 444 yds/game against an official 344	Playoff plays counted in the numerator against a 17-game regular-season denominator	All rates and records filter `season_type == "REG"`. Postseason games still get replays.
4	30 of 32 teams had no depth chart	The legacy weekly schema’s *maximum* week covers only teams still in the playoffs	Take the last *regular-season* week. Both the pre-2025 and post-2025 schemas are handled and normalise to one shape.
5	8 teams per season had an empty draft class	The draft dataset uses Pro-Football-Reference abbreviations (`GNB`, `KAN`, `NWE` …)	`ABBR_FIXES`, which also covers relocations. Every dataset is normalised to the schedule’s vocabulary at the boundary.
6	1,789 KiB of images to paint 32 logos at 40px	nflverse stores ESPN’s 500×500 master, 42–79 KB each	`logoAt` asks ESPN’s combiner for 2× the CSS size. Page weight 1,789 → 262 KiB.
7	A.J. Brown’s 36px thumbnail was a **3.8 MB PNG**	55 of 2,269 headshot URLs use `/image/private/`; the first regex matched only `/image/upload/`. The E2E check ran against `?q=allen`, narrow enough to contain none of them	One-word regex fix; `headshotAt` gained unit tests; the E2E test now loads the *unfiltered* listing with byte budgets; `validate.py` rejects any stored headshot the frontend cannot resize.
8	Every first visit downloaded 26 kB of Framer Motion, including game pages that never animate	It was imported in the shared layout for one page-transition fade	The transition is nine lines of CSS. Motion is confined to two chunks via `LazyMotion ... strict`, which makes a heavier `motion.*` import a runtime error.
9	60 forced synchronous layouts per second during playback	`getBoundingClientRect()` called inside the frame, while the scrubber dirtied the DOM every frame by writing its own progress	Measure in a `ResizeObserver` and pass the size in. Forced layouts: 60/s → 0.
10	The scrubber reported a different play from the readout beside it	The sub-pixel write-skipping optimisation deferred a write that straddled a play boundary	The skip test gained a second term: skip only if *both* the thousandth-of-track and the floor-of-cursor are unchanged. Test named after the bug.
11	Pressing Restart dropped keyboard focus to `<body>`	Restart disabled itself on reaching play 0 — which pressing it does. A disabled element cannot hold focus	Restart is never disabled. Restarting an already-restarted replay is harmless. Test named after the bug.
12	Aiming at each of 41 key-play markers hit a different play **19 times**	Markers cluster to a median of 3 px apart on a phone; stacked 24px hit boxes hand the click to whichever is later in the DOM	The rail resolves clicks to the nearest marker in the capture phase, with `event.detail === 0` excluded so keyboard activation still acts on the focused marker. The E2E test clicks a *pixel*.

13	Typing “garrett” left “t” in the search box	The input was driven from the query string. `setSearchParams` is a navigation, so each keystroke read a filter from a render that had not happened yet	The box owns its state; the URL mirrors it. The two `<select>` facets stay URL-driven — one change per interaction cannot race itself.
14	The deployed site served the wrong `Cache-Control` for game files	Vercel `headers` are cumulative, not first-match-wins. A plain `/data/(.*)` rule silently overwrote the game rule	Negative lookahead in the second rule. Invisible locally — `vite preview` reads no `vercel.json` — and only caught by curling the deployment.
15	A 0.2 KB file could take the whole dashboard down	`meta.json` was in the same `Promise.all` as the load-bearing files	`getMeta().catch(() => null)` inside `Promise.all`, so it settles with the others (no layout shift) but cannot fail the page. An E2E test aborts that request.
16	A 2020 replay’s breadcrumb dropped the visitor on this year’s Chiefs	The season lived only on the team page, not in any link out of it	Every link into a team page carries the season it means. `resolveSeason` is one function used by both the control and the fetch.
17	Pressing a season tab by keyboard dropped focus to `<body>`	The control was rendered inside the loaded content, so it unmounted while the next season’s file arrived	The breadcrumb and the control render from `meta.json`, which does not change between seasons; only the body swaps for a skeleton. Its accessible name is “Season”, not the team’s, so it does not rename itself under the focus.
18	60+ false contrast failures	Tailwind v4 serialises colours as `oklch()`; a regex over the stylesheet cannot resolve them	Paint each colour into a canvas and read it back. Found two *real* failures (2.46:1 and 4.05:1) that the regex noise had buried.
19	A NaN cursor made the entire curve disappear silently	`Math.min(last, Math.max(0, NaN))` is still `NaN`, and canvas skips non-finite coordinates without complaint	`clampCursor` checks `Number.isFinite` first and returns 0. Test: “a non-finite cursor still draws”.
20	Both Los Angeles clubs and both New York clubs collapsed to the same string	The short name was produced by truncation	`teamNickname` takes the last token: every NFL club name ends in a unique single-word nickname.

HOW TO TELL ONE OF THESE

Use bug #12 or #7. Both have the same excellent shape: **a measurable symptom, a cause that is not where you would look first, a fix that is small, and a change to the process that makes the class of bug impossible to reship.** For #7 the process change is the best part — the bug shipped because the E2E test’s query was narrow enough to miss the affected rows, so the test now loads the unfiltered listing with explicit byte budgets, and the validator rejects any URL shape the frontend does not know how to resize.

SECTION EIGHTEEN

Trade-offs and limitations

18.1 Decisions, with their costs stated

DECISION	BENEFIT	COST
Static JSON over a backend or API	No server, no database, no cold start, no runtime failure mode more complex than a 404, and a CDN that can cache everything	A 15 MiB repository and a manual (now scheduled) re-run when a new week lands. *For a dataset that is finished, this is the right shape.*
Local ETL over runtime processing	Parsing 273,623 plays takes minutes; doing it per request would be absurd, and doing it in a serverless function would reintroduce the backend the design exists to avoid	Data freshness is a human action — mitigated, not removed, by the Tuesday workflow
Imperative animation state over React state	The frame budget is spent on drawing: 0.14 ms of 16.7 ms, 0 dropped frames in 239	The replay's cursor is not inspectable in React DevTools and cannot be driven by a React render
`useSyncExternalStore` over a ref + forced re-render	Correct under concurrent rendering, rather than relying on the render phase seeing a mutation it should not	More ceremony: a subscribe/read/write triple per published value
One file per team per season	Every page describes one year; the fetched file got *smaller* (5.9 → 3.8 KB gzipped)	32 files became 192; the repository grew 90 → 99 MB
Runtime contract validation	A wrong file names the exact path that disagreed, at the fetch that caused it; no type assertion anywhere else in the app	1.17 kB gzipped and 0.23 ms on the largest file. Both measured; both trivial.
Search index with headshots (73 KB vs 35 KB)	A list of two thousand names is scanned by face, not by name	`/players` is the heaviest route. Thumbnails load lazily; dropping them is a one-line change if it stops being worth it.

18.2 Recharts first, then a custom canvas

The win-probability chart was built in **Recharts** in an earlier stage — deliberately, as a way to validate the data before investing in the flagship. It did that job and was then deleted.

A charting library was never going to give per-frame control of a 218-point animated reveal, and shipping both would have cost the bundle twice. **Validating first and replacing second is cheaper than either guessing or building the hard thing twice.**

18.3 Known limitations, stated plainly

- **No future fixtures.** `GameSummary.homeScore` is required, and an unplayed game has no score. Making them optional is a contract change, not a new file.
- **The comparison is of the displayed season only.** Both sides come from `meta.displaySeason`. `getTeamSeason` already takes a season, but a chosen year needs its own control on that page to avoid being invisible state in a URL.
- **863 rostered players have no page.** By design — they have no recorded production, and a page for them would be their roster row with a photograph on it. `hasProfile` means a link only ever points at a page that exists.
- **Expected wins is market-implied, not a Vegas over/under.** The source dataset was discontinued after 2020. The dashboard says so on the page.
- **The replay chart series is recomputed on every mount** rather than precomputed in the ETL. Currently 0.05 ms for a typical game, so this is a correctness-of-ownership argument rather than a performance one.

- **No visual regression testing on the canvas.** The drawing code is pure and deterministic, so a pixel snapshot per game archetype would catch what a recording context cannot.

18.4 What I'd build next

Compare two games, or two seasons, on one axis
The chart already draws from a pure series; a second series is mostly a legend and a colour decision.

A drive-level layer over the play-level one
The ETL already knows possession and scoring; grouping plays into drives would let the replay skip forward a drive at a time, which is closer to how people actually talk about a game.

Search across all 1,695 games
By team, week, margin, or "games that swung more than 60 points". `games-index.json` already exists, is validated, and has a typed accessor. At 27 KB gzipped this is a filter over one fetch, not a backend.

A shareable deep link into a moment — `/game/<id>?play=142`
The replay's single cursor already makes this almost free.

Precompute the replay's per-play deltas in the ETL
So the client stops recomputing the chart series on every mount. An ownership argument, not a performance one.

Visual regression tests on the canvas
A pixel snapshot per game archetype — blowout, comeback, overtime thriller — would catch what a recording context cannot.

18.5 Attribution and licensing

Data from **nflverse**, used under **CC BY 4.0**. nflverse data is itself derived from NFL play-by-play sources; the win-probability model is nflfastR's. Team logos are served from ESPN's CDN, the URLs supplied by the nflverse teams dataset.

The attribution the licence requires is in the footer of every page, not buried in a README. The project is not affiliated with, endorsed by, or connected to the NFL or any of its clubs; team names, logos and colours are the property of their respective owners and appear for identification only.

SECTION NINETEEN

Interview Q&A

Thirty questions this project invites, with answers you can give in under ninety seconds each. The strongest answers name a number or a bug.

19.1 Architecture and design

Why no backend?

Because the data is historical and immutable. A finished season is a fact, so there is nothing to serve dynamically. Declaring that removes the database, the API layer, cache invalidation, cold starts and every runtime failure mode more complex than a 404 — and replaces them with a build step. The cost is a 15 MiB repository and a re-run when a new week lands, which is now a scheduled workflow that opens a pull request. For a dataset that is finished, static is the right shape. If the product needed live in-game data, that trade flips entirely.

How would this scale to 1999–2026 instead of 2020–2026?

The per-visit cost does not change at all — a visitor fetches one team-season file and one game file regardless of how many exist. What grows is the repository (roughly 13 MB of replays per season) and two index files: `games-index.json` at ~26 KB gzip per six seasons, and `standings.json` at ~2.6 KB per six. At twenty-seven seasons the games index would be ~120 KB gzipped, which is the first thing I would page or shard — probably one index file per season, since the comparison page is its only reader and it already knows which seasons it wants. Git would be the real constraint before the CDN is.

Why commit 99 MB of generated data to git?

So that a clone is a working application: no Python, no nflverse round trip, no build step before `npm run dev`. It also makes every deploy reproducible and every data change reviewable as a diff — which is the entire mechanism of the weekly refresh. It packs to about 15 MiB. If it kept growing I would move the game layer to a release artifact or object storage and keep the small indices in git, but at this size the reviewability is worth more than the bytes.

Why is the team layer one file per team per season rather than one per team?

Because every figure on a team page — the record, the roster, the depth chart, the stat lines — describes exactly one season, and the old shape had six seasons of games sitting under the newest season's squad. The counter-intuitive part is that splitting made the fetched file *smaller*: 5.9 KB gzipped to 3.8 KB, because a per-season file no longer carries six seasons of games to make them reachable. 32 files became 192 and the repo grew 9 MB; the per-visit cost went down.

Why is `standings.json` separate from `teams-index.json`?

Two reasons. First, they answer different questions: the index is "who are these teams now" (colours, division), standings is "what happened in a given year". Second, cost allocation. The dashboard needs every season at once so a season switch is a re-render rather than a fetch — 2.6 KB for all 192 rows. Putting that on `teams-index.json` would have charged the comparison page and every team page for it too, and they never look at it.

19.2 The replay

Walk me through the animation loop.

One `requestAnimationFrame` loop, one fractional play cursor in a ref. Each frame: take the timestamp delta, clamp it to [0, 100ms], advance the cursor by `delta/1000 × 8 plays/sec × speed`, repaint the canvas, write the scrubber thumb, and publish `Math.floor(cursor)` through `useSyncExternalStore`. That publish is a no-op unless the floor actually changed, so React re-renders about eight times a second while the canvas repaints sixty. Reaching the end clamps, publishes, cancels and sets playing false.

Why not `useState` for the cursor?

Because it would mean a render, a reconciliation and a commit sixty times a second to move a few hundred pixels of line. The canvas is not React's to reconcile — it is one element whose contents I paint imperatively. So the cursor lives where it belongs, in a ref, and React is told only the two facts a person can see: whether it is running, and which play it is on.

Why `useSyncExternalStore` rather than a ref plus `forceUpdate`?

Because the loop genuinely *is* an external system on a different clock, and that is exactly what the hook is for. A forced re-render works today by relying on the render phase observing a mutation it is not supposed to see, which is not guaranteed under concurrent rendering. Using React's own escape hatch makes it correct by contract rather than by luck — and it gets tearing protection for free.

Why is the drawing module pure?

Because a frame is fully determined by the cursor. Once nothing in the drawing code reads the clock, React state or the DOM, the same function renders the running animation, a paused frame, and any position a scrubber is dragged to — and it can be tested against a recording context with no browser at all. That is how `indexAtOffset` inverting `xForIndex` gets asserted for every play at three widths in a unit test.

What happens when the tab is backgrounded?

`requestAnimationFrame` stops firing, so the first frame after returning carries the whole gap — potentially minutes. Uncapped, that jumps the replay straight to the final whistle. `MAX_FRAME_MS` is 100, six frames' worth, so a hidden tab simply holds its place. There is a unit test that feeds a 120-second frame and asserts the cursor advances 0.8 of a play. The delta is clamped at the low end too, because a frame's timestamp can predate the one that scheduled it, which would run the replay backwards.

How do you keep 60fps?

Three things, all measured. Nothing reads layout during a frame — the canvas size comes from a `ResizeObserver`, which took forced layouts from sixty a second to zero. The scrubber write is skipped when neither the thousandth-of-track nor the play index changed. And React renders eight times a second instead of sixty. Result: median frame 16.70 ms over 240 consecutive frames of a 218-play overtime game, zero dropped, zero long tasks.

Did you cache the static background to an offscreen canvas?

No — I measured it first. All canvas drawing costs 0.14 ms per frame and the axis labels 0.03 ms, against a 16.7 ms budget. Blitting a cached bitmap of the same size costs more than redrawing. It is the optimisation that looks obviously right and is measurably wrong, which is why the reasoning is a comment above `drawBackground` rather than a deleted branch.

19.3 Types and data integrity

Why validate at runtime if the data is generated by your own ETL?

Because `response.json()` returns `unknown` and TypeScript cannot follow it across a network boundary. The failures it catches are real: a stale deploy reading a newer file shape, a half-written ETL run, an SPA fallback serving HTML with a 200. Without it, the symptom is `undefined` surfacing three components away from the fetch that caused it. With it, the message is `game.plays[0].homeWinProb: expected a finite number, received "0.5"`. It costs 1.17 kB gzipped and 0.23 ms on the largest file — both measured.

Isn't validating twice — Python and TypeScript — duplication?

They catch different failures. The Python validator catches a bad *generation* before it is committed, and it can check things the browser cannot: that every team game reference resolves to a file on disk, that season directories match what meta advertises, that plays are ordered. The TypeScript one catches a bad *delivery* — wrong file, stale deploy, truncated response. And the duplication is itself guarded: `validate.py` parses `src/types/nfl.ts` and fails on drift, because a validator that has quietly stopped matching the contract is worse than none.

Why omit unknown fields instead of emitting `null`?

Because `null` does not satisfy `number | undefined` under `strict`, so omission is the only representation that typechecks — and it keeps "missing" spelled exactly one way. The semantic reason matters more: a kickoff has no down, and `down: null` invites a UI to render a placeholder that looks like data. The ETL drops the key, the contract marks it `?`, the validator fails on a `null` in an optional slot, and the parser accepts `undefined` only.

How do you stop the contract from drifting across three languages?

`src/types/nfl.ts` is the single source of truth and the file says so: "If you change one, change all three in the same commit." On the TypeScript side, parsers return the interfaces *by annotation*, so adding a property is a compile error until it is checked. On the Python side, `validate.py` regex-parses the interfaces and reports three drift kinds: a field only the validator has, a field only `nfl.ts` has, and a field whose optionality disagrees. Neither side can silently fall behind.

19.4 Data engineering

What was the hardest part of the ETL?

Trusting nothing. Three separate ordering and freshness defects in the play stream alone: `play_id` is not chronological, timeout rows carry stale scores, and playoff plays inflate per-game rates. None announces itself — each just produces a plausible-looking wrong number. The fix in each case was to state the invariant in the validator, not just in the transform, so the same class of defect fails the build next time.

How do you handle an upstream schema change?

It happened: nflverse changed the depth-chart schema for 2025, from weekly snapshots to dated ones. Both are handled in one function that normalises to a single shape, chosen by a column presence check rather than a season number, so a backfill of an old season still works. The wider answer is the pin — `nflreadpy==0.1.5` exactly, because it is pre-1.0 and a floating range would make the pipeline silently non-reproducible.

Why polars and not pandas?

`nflreadpy` returns polars frames natively, so pandas would mean converting frames back and forth for no benefit. The project brief specified pandas; I documented the deviation rather than following it silently. The expensive operation here is loading a 300-column play-by-play frame per season, and it is loaded exactly once and reused for both the game layer and the team stat lines — loading it twice would be the most expensive thing this script could do.

How do you keep a weekly automated refresh safe?

It fails closed at four gates and stops short of deploying. The refresh rebuilds only the newest season; `merge_index` aborts if any game in the old index is missing from the rebuild; `validate.py` must pass; and the frontend's own unit tests and build must pass, because if the shipped app can no longer read the shipped data that is the same failure as an invalid file. Then it opens a pull request — not a push. The diff is the review. And a run with no changes opens no PR, because a weekly "no change" PR trains people to merge without looking.

How does the pipeline decide which season to show?

A named constant, not a judgement call: `PROMOTE_MIN_PLAYED = 64`, roughly four weeks. Below that the team layer stays on the last complete season. The reason is that a two-game season does not degrade gracefully — it collapses: four of thirty-two teams would have had a record, and a projected-wins figure of 0.6 would sit next to a last-season record of 11-6. `--display-season` overrides it by hand, and a separate guard refuses to publish a dashboard missing teams whatever the flag says.

19.5 Frontend craft

Why is loading derived rather than stored in `useAsync`?

Because it removes a race instead of managing one. The settled result is tagged with the key it belongs to, and the hook returns loading whenever that tag does not match the current key. A slow request for one team can no longer paint over a fast one for the next — there is nothing to cancel, because the answer to "whose result is this?" is in the result. It also avoids a redundant render on every key change.

Why does the cache store promises rather than values?

So that two components asking for the same file in the same tick share one request — the team page and the footer both want `meta.json`. Storing values would let both fire. And rejected entries are evicted, so a failure is retryable rather than cached forever. It stores the unchecked JSON rather than a typed value because the cache cannot know which parser filled an entry, so handing it back typed would need an assertion on every hit; re-validating costs 0.23 ms and keeps the no-assertions invariant.

How do you handle errors in the UI?

Three kinds, because those are the three a UI can meaningfully branch on. `not-found` gets a way back and no retry button — retrying a 404 is theatre. `network` gets a retry that issues a genuinely new request. `malformed` means the data is wrong, and the message names the exact contract path. The error state renders an `h1`, because it replaces the page it was rendered for and the document would otherwise have no heading.

Why are skeletons route-shaped instead of a spinner?

Because a spinner followed by a differently-shaped page is two waiting states for one click. The Suspense fallback reads the pathname and renders the same skeleton the page itself would render, so there is one continuous shape from click to content. It also reserves 120vh, which fixed most of the measured layout shift: every route in this app loads to more than one screen, so the space is genuinely used — that is the job of a skeleton, not padding to move a number.

How did you get CLS to 0?

Two fixes, both self-inflicted. The app shell is `min-h-dvh flex flex-col`, so a short skeleton left the footer visible and the real content then shoved it off-screen — skeletons now reserve 120vh. And the dashboard's coverage note arrived a frame after the standings and pushed 50 px down, so the three files it needs now settle together with the freshness one allowed to fail. Delaying `meta.json` by two seconds against the production build now measures 0.0000.

Why is the search input not URL-driven when everything else is?

Because `setSearchParams` is a navigation, and a fast typist's keystroke handlers each read a filter from a render that has not happened yet. Typing "garrett" left "t" in the box. So the input owns its value and the URL mirrors it. The two `<select>` facets stay URL-driven, because one change per interaction cannot race itself. The asymmetry is deliberate and commented.

19.6 Accessibility and testing

How did you approach accessibility?

Keyboard first, measured second, automated third. Lighthouse scores 100 on all five routes on both form factors, but that is a floor. The real audit was tabbing every page — 44 stops on the League page, 30 on Team, the replay fully operable at 320px — and measuring contrast from the rendered page by painting colours into a canvas and reading them back, because Tailwind v4 emits `oklch()` and a regex produced sixty false failures. That found two real text failures at 2.46:1 and 4.05:1, and nine team colours below 3:1 on the replay card.

How do you make 32 team colours accessible without making them all grey?

Lift, don't replace. `legibleOn` mixes the hue toward white in 2% steps until it clears 3:1 against the actual composited surface, so every team keeps its identity. For bar fills, `teamAccent` tries the team's own *secondary* colour before lightening — Pittsburgh gets its gold rather than a pale black. Both are enforced by tests that read the real team index, because nine and twelve clubs respectively failed before those tests existed.

What is your testing philosophy?

Test behaviour that could plausibly break, not lines. Two of the tests are named after the bug rather than the function — "keeps Restart enabled, so pressing it never drops focus to the body" — which tells the next reader why the line below cannot be simplified. Several suites read the real generated JSON rather than fixtures, so a data refresh that breaks the app fails the unit suite. And the E2E suite runs against the production build on `vite preview`, because that server reproduces a failure mode the dev server does not.

Tell me about a test that was wrong.

The headshot E2E check ran against `?q=allen` — narrow enough to contain none of the 55 players whose image URL used a different delivery path. It passed while the default listing shipped 775 KB of images, one of them a 3.8 MB PNG in a 36-pixel box. The fix was a one-word regex change; the interesting part is that the test now loads the *unfiltered* listing with explicit byte budgets, and `validate.py` rejects any stored URL shape the frontend does not know how to resize. A narrow fixture is a test that agrees with you.

What would you do differently?

Three things. I would write `headshotAt`'s unit tests before shipping it rather than after the bug — it was the one function in `lib/` with no tests, and it was the one that failed. I would have added visual regression snapshots on the canvas from the start, since the drawing code is pure and deterministic and that is nearly free. And I would precompute the replay's per-play deltas in the ETL, not for the 0.05 ms but because the client recomputing a derived series is the wrong owner for it.

SECTION TWENTY

Appendices

A. Numbers cheat-sheet

If you remember nothing else, remember these. Every one is measured.

FACT	NUMBER
Games with replays / plays processed	1,695 / 273,623
Generated JSON files	4,166
Average game file, gzipped	6.6 KB
Team-season file, gzipped (was 5.9 KB before the split)	3.8 KB
All 192 standings rows, gzipped	2.6 KB
Cost of changing season on the dashboard	0 bytes
Entry chunk, gzipped	84.78 kB
Replay engine chunk, gzipped — game pages only	6.29 kB
Framer Motion, gzipped — League and Team only	25.80 kB
Runtime contract: added bundle / validation of the 239 KB index	1.17 kB / 0.23 ms
`JSON.parse` on the same file, for comparison	0.57 ms
Median frame during playback (240 frames, 218-play OT game)	16.70 ms
Dropped frames / long tasks	0 / 0
All canvas drawing per frame / axis labels	0.14 ms / 0.03 ms
Forced layouts during playback, before → after	60/s → 0
React renders during playback at 1×	8/s, not 60/s
Plays revealed per second (`PLAYS_PER_SECOND`)	8
Max cursor advance per frame (`MAX_FRAME_MS`)	100 ms
Key-play threshold (`KEY_PLAY_WP_SWING`) / share of plays flagged	0.10 / ~9.5%
Key plays per game: average / maximum	15 / 41
Season promotion threshold (`PROMOTE_MIN_PLAYED`)	64 games
Margin sigma for spread → win probability	13.86
League page image weight, before → after `logoAt`	1,789 → 262 KiB
A.J. Brown's profile, before → after the regex fix	3.8 MB → 6.1 KB
Default `/players` listing: headshots / bytes after the fix	51 / 80 KB
Team colours failing 3:1 on the card / on the bar track	9 / 12
Distinct text styles audited, all clearing 4.5:1	88
Tab stops: League / Team	44 / 30
Tests: unit & component / E2E specs / fault injections	389 / 40×2 / 16
Lighthouse desktop, every route, every run	100
CLS, every route, both forms	0

Player profiles / rostered players	2,274 / 3,137
Linemen without a profile / with one	346 / 204

B. Commands

Frontend

```
npm install
npm run dev            # http://localhost:5173

npm run typecheck      # tsc -b, four projects: app, node, tests, e2e
npm run lint           # oxlint
npm run format         # prettier --write .
npm test               # Vitest, 389 tests
npm run test:coverage  # with a v8 coverage report
npm run e2e            # Playwright (builds and serves the app itself)
npm run build          # typecheck + production build
npm run verify         # everything CI runs, in order

npx playwright install chromium   # once, before the first e2e run
```

ETL — only needed to regenerate or extend the dataset

```
cd etl
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt      # nflreadpy, polars, pyarrow — pinned exactly

python build_data.py                 # full build: ../public/data, 99 MB, a few minutes
python build_data.py --refresh       # newest season only, merged into the index
python build_data.py --refresh --display-season 2026   # force the team layer
python validate.py --data ../public/data
python test_refresh.py               # fault-inject the guardrails
```

Measuring

```
npx lighthouse https://nfl-season-hub.vercel.app/ --preset=desktop --view
npm run build && npm run preview -- --port 4173    # then point Lighthouse at it
```

Lighthouse is not a dependency of this project — it is run with `npx` when a measurement is wanted, so nobody pays 21 MB of install for a number.

C. Where things live

IF YOU ARE ASKED ABOUT…	OPEN
The data contract	`src/types/nfl.ts`
Runtime validation	`src/lib/contract.ts`
Fetching, caching, error kinds	`src/lib/data.ts`
The React data bridge	`src/lib/useAsync.ts`
The animation loop	`src/lib/useReplay.ts`
Canvas drawing and geometry	`src/lib/replayCanvas.ts`
Chart series, quarter marks, key plays	`src/lib/winprob.ts`
Contrast and team colours	`src/lib/colors.ts`

Image resizing	`src/lib/logos.ts`
Sorting, filtering, URL state	`src/lib/league.ts`
Search ranking and normalisation	`src/lib/playerSearch.ts`
Season rules and coverage wording	`src/lib/season.ts`
The transport UI and keyboard map	`src/components/WinProbCanvas.tsx`
Roving tabindex and nearest-marker clicks	`src/components/KeyPlayRail.tsx`
The pipeline	`etl/build_data.py`
Unattended refresh decisions	`etl/refresh_rules.py`
Contract enforcement and drift detection	`etl/validate.py`
Cache tiers and the SPA fallback	`vercel.json`
The three CI jobs	`.github/workflows/ci.yml`
The Tuesday refresh	`.github/workflows/refresh-data.yml`

D. Glossary

TERM	MEANING IN THIS PROJECT
nflverse	The open-source NFL data project publishing play-by-play, rosters, schedules and more. CC BY 4.0.
nflfastR	The R package whose win-probability model produces the `home_wp` values this site animates.
EPA	Expected Points Added — the change in a team's expected points from one play. The core efficiency metric here; `successRate` is the share of plays with positive EPA.
Explosive rate	Share of plays gaining 20+ yards.
Closing spread	The final betting line before kickoff. Positive `spread_line` means the home team is favoured. Converted to a win probability via the normal CDF at σ = 13.86.
Expected / projected wins	Those per-game win probabilities summed across a regular season. *Market-implied*, not a Vegas over/under — that dataset was discontinued after 2020.
Key play	A play flagged by the ETL: a score, a turnover, a made field goal, or a win-probability swing of ≥ 0.10. ~9.5% of plays.
Display season	The season the team layer describes — rosters, stat lines, projections. Decided by `PROMOTE_MIN_PLAYED`, not by whoever ran the build.
Team layer	The per-team, per-season files plus the standings rows. A season gets one only once it is complete or past the promotion threshold.
The cursor	The single fractional play index that is the replay's only notion of position. A ref, not state.
Roving tabindex	A group of controls exposed as one tab stop, with arrow keys moving focus inside it. Used for the 41-marker key-play rail.
CLS / LCP / TBT / FCP	Cumulative Layout Shift, Largest Contentful Paint, Total Blocking Time, First Contentful Paint — the Core Web Vitals Lighthouse reports.
`stale-while-revalidate`	A cache directive letting a CDN serve a stale response while fetching a fresh one. Used instead of `immutable` because game files are not content-hashed and do change.

E. Project timeline, from the commit history

STAGE	WHAT LANDED

Stage 1	The ETL and the flat data contract, plus a checkpoint commit fixing contract defects found by audit
Stage 2	App shell, routing and the typed data layer; checkpoint fixed a dead retry path and Suspense placement
Stage 3	League dashboard; checkpoint replaced an entrance spring with a tween
Stage 4	Team page; checkpoint fixed count-up accessibility and a negative-value frame
Stage 5A–5E	The replay, built in five passes: a static Recharts chart for data validation → the canvas and rAF loop → scrubber and transport → game context and key plays → a performance pass. Four checkpoint commits, each named after the bug it fixed: plot play order not elapsed time; stop Restart stealing its own focus; never defer a scrubber write across a play boundary; resolve key-play clicks to the nearest marker.
Stage 6	Polish, accessibility and cross-cutting UX; checkpoint fixed text contrast across the app
Stage 7	Engineering credibility layer; checkpoint fixed a red build, a missed strict flag and dead code
Stage 8	Compare, the controlled data-refresh process, player pages, the player search, six seasons everywhere, the CLS fixes, and the season-continuity work

42 commits. **Roughly a third of them are checkpoint commits that fix something the previous commit got wrong** — which is worth pointing at, because it is the honest record of how the work actually went, and every one of those fixes is a story in Section 17.

Data from nflverse, CC BY 4.0. Not affiliated with the National Football League.